```
[00:02:38] Extracting lib32... done
[00:02:42] Cleaning up... done
[00:02:42] Recording filesystem state for clean... done
[00:02:42] Upgrading using ftp
/etc/resolv.conf -> /poudriere/jails/13amd64/etc/resolv.conf
Looking up update.FreeBSD.org mirrors... 3 mirrors found.
Fetching public key from update4.freebsd.org... done.
Fetching metadata signature for 13.1-RELEASE from update4.freebsd.org... done.
Fetching metadata index... done.
Fetching 2 metadata files... done.
Inspecting system... done.
Preparing to download files... done.
Fetching 124
patches.....10....20....30....40....50....60....70....80....90....100....110....120..
done.
Applying patches... done.
Fetching 6 files... done.
The following files will be added as part of updating to
13.1-RELEASE-p1:
/usr/src/contrib/unbound/.github
/usr/src/contrib/unbound/.github/FUNDING.yml
/usr/src/contrib/unbound/contrib/drop2rpz
/usr/src/contrib/unbound/contrib/unbound_portable.service.in
/usr/src/contrib/unbound/services/rpz.c
/usr/src/contrib/unbound/services/rpz.h
/usr/src/lib/libc/tests/gen/spawnp_enoexec.sh
The following files will be updated as part of updating to
13.1-RELEASE-p1:
[…]
Installing updates...Scanning //usr/share/certs/blacklisted for certificates...
Scanning //usr/share/certs/trusted for certificates...
 done.
13.1-RELEASE-p1
[00:04:06] Recording filesystem state for clean... done
[00:04:07] Jail 13amd64 13.1-RELEASE-p1 amd64 is ready to be used
```

```
# poudriere ports -c -p local -m git+https
[00:00:00] Creating local fs at /poudriere/ports/local... done
[00:00:00] Checking out the ports tree... done
```

On a single computer, poudriere can build ports with multiple configurations, in multiple jails, and from different port trees. Custom configurations for these combinations are called *sets*. See the CUSTOMIZATION section of poudriere(8) for details after ports-mgmt/poudriere or ports-mgmt/poudriere-devel is installed.

The basic configuration shown here puts a single jail-, port-, and set-specific `make.conf` in `/usr/local/etc/poudriere.d`. The filename in this example is created by combining the jail name, port name, and set name: `13amd64-local-workstation-make.conf`. The system `make.conf` and this new file are combined at build time to create the `make.conf` used by the build jail.

Packages to be built are entered in `13amd64-local-workstation-pkglist` (ports with FLAVORS can be defined with @FLAVOR):

```
editors/emacs
devel/git
devel/php-composer2@php82
ports-mgmt/pkg
...
```

Options and dependencies for the specified ports are configured:

```
# poudriere options -j 13amd64 -p local -z workstation -f 13amd64-local-workstation-
pkglist
```

Finally, packages are built and a package repository is created:

```
# poudriere bulk -j 13amd64 -p local -z workstation -f 13amd64-local-workstation-
pkglist
```

While running, pressing `Ctrl` + `t` displays the current state of the build. poudriere also builds files in `/poudriere/logs/bulk/jailname` that can be used with a web server to display build information.

After completion, the new packages are now available for installation from the poudriere repository.

For more information on using poudriere, see poudriere(8) and the main web site, https://github.com/freebsd/poudriere/wiki.

4.6.2. Configuring pkg Clients to Use a poudriere Repository

While it is possible to use both a custom repository along side of the official repository, sometimes it is useful to disable the official repository. This is done by creating a configuration file that overrides and disables the official configuration file. Create `/usr/local/etc/pkg/repos/FreeBSD.conf` that contains the following:

```
FreeBSD: {
    enabled: no
}
```

Usually it is easiest to serve a poudriere repository to the client machines via HTTP. Set up a webserver to serve up the package directory, for instance: `/usr/local/poudriere/data/packages/13amd64`, where `13amd64` is the name of the build.

If the URL to the package repository is: `https://pkg.example.com/13amd64`, then the repository configuration file in `/usr/local/etc/pkg/repos/custom.conf` would look like:

```
custom: {
    url: "https://pkg.example.com/13amd64",
    enabled: yes,
}
```

If exposing the package repository to the internet is not desired, the `file://` protocol can be used to point to the repository directly:

```
custom: {
    url: "file:///usr/local/poudriere/data/packages/11amd64",
    enabled: yes,
}
```

4.7. Post-Installation Considerations

Regardless of whether the software was installed from a binary package or port, most third-party applications require some level of configuration after installation. The following commands and locations can be used to help determine what was installed with the application.

- Most applications install at least one default configuration file in `/usr/local/etc`. In cases where an application has a large number of configuration files, a subdirectory will be created to hold them. Often, sample configuration files are installed which end with a suffix such as `.sample`. The configuration files should be reviewed and possibly edited to meet the system's needs. To edit a sample file, first copy it without the `.sample` extension.
- Applications which provide documentation will install it into `/usr/local/share/doc` and many applications also install manual pages. This documentation should be consulted before continuing.
- Some applications run services which must be added to `/etc/rc.conf` before starting the application. These applications usually install a startup script in `/usr/local/etc/rc.d`. See Starting Services for more information.



By design, applications do not run their startup script upon installation, nor do they run their stop script upon deinstallation or upgrade. This decision is left to the individual system administrator.

- Users of csh(1) should run `rehash` to rebuild the known binary list in the shells `PATH`.
- Use `pkg info` to determine which files, man pages, and binaries were installed with the application.

4.8. Dealing with Broken Ports

When a port does not build or install, try the following:

1. Search to see if there is a fix pending for the port in the Problem Report database. If so,

implementing the proposed fix may fix the issue.

2. Ask the maintainer of the port for help. Type `make maintainer` in the ports skeleton or read the port's `Makefile` to find the maintainer's email address. Remember to include the output leading up to the error in the email to the maintainer.



Some ports are not maintained by an individual but instead by a group maintainer represented by a mailing list. Many, but not all, of these addresses look like freebsd-listname@FreeBSD.org. Please take this into account when sending an email.

In particular, ports maintained by ports@FreeBSD.org are not maintained by a specific individual. Instead, any fixes and support come from the general community who subscribe to that mailing list. More volunteers are always needed!

If there is no response to the email, use Bugzilla to submit a bug report using the instructions in Writing FreeBSD Problem Reports.

3. Fix it! The Porter's Handbook includes detailed information on the ports infrastructure, enabling the fixing of occasional broken ports or the submission of new ones. Install the package instead of the port using the instructions in Using pkg for Binary Package Management.

Chapter 5. The X Window System

5.1. Synopsis

An installation of FreeBSD using bsdinstall(8) does not automatically install a graphical user interface. This chapter describes how to install and configure the Xorg(1) server, which provides the open source X Window System (colloquially X11) used to provide a graphical environment.

Before reading this chapter:

- Know how to install additional third-party software as described in Installing Applications: Packages and Ports.

Read this chapter to learn:

- How to select and install drivers for the graphics processor (GPU).
- The various components of the X Window System, and how they interoperate.
- How to install and configure the X.org server.
- How to install fonts for the X Window System.

5.2. Graphics Drivers

***abstract**: Identify the GPU, the port providing a driver for it, install it, then enable it to run at subsequent boot with sysrc(8).*

Before FreeBSD can render a graphical environment, it needs a kernel module to drive the graphics processor. Graphics drivers are a fast-moving, cross-platform target, which is why this is developed and distributed separately from the FreeBSD base system.

The following table shows the different graphics processors supported by FreeBSD, their corresponding module, and which port provides it:

Table 8. Supported Graphics Devices

Type	License	Module	Port
Intel®	Open Source	`i915kms`	graphics/drm-kmod
AMD®	Open Source	`amdgpu` or `radeonkms`	graphics/drm-kmod
NVIDIA®	Proprietary	`nvidia-drm`, `nvidia-modeset`, or `nvidia`	graphics/nvidia-drm-kmod or x11/nvidia-driver
System Console Framebuffer	Open Source	`scfb`	x11-drivers/xf86-video-scfb
VESA BIOS Extension	Open Source	`vesa`	x11-drivers/xf86-video-vesa

Type	License	Module	Port
VirtualBox®	Open Source	vboxvideo	emulators/virtualbox-ose-additions
VMware®	Open Source	vmwgfx	x11-drivers/xf86-video-vmware

There are several generations of driver technologies supported.

- Direct Rendering drivers allowing PRIME offloading. PRIME allows for multiple providers of graphics processing to coexist. PRIME is described further in Graphics Configuration.
- Kernel Modesetting (KMS) This allows the driver to directly specify the display mode. This is required to support suspend and resume when using the vt(4) console driver.
- User Modesetting The oldest class of drivers is still supported, however they may only be used with sc(4) console and older versions of the Xorg(1) graphical environment.

The following command can be used to identify which graphics processor is installed in the system:

```
% pciconf -lv | grep -B3 display
```

The output should be similar to the following:

```
vgapci1@pci0:0:2:0:     class=0x030000 rev=0x0c hdr=0x00 vendor=0x8086 device=0x46a6
subvendor=0x1028 subdevice=0x0b29
    vendor     = 'Intel Corporation'
    device     = 'Alder Lake-P GT2 [Iris Xe Graphics]'
    class      = display
```

Detailed instructions on installing and enabling these drivers are in the subsequent subsections.



If the graphics processor is not supported by Intel®, AMD®, or NVIDIA® drivers, then SCFB or VESA modules should be used. SCFB module must be used when booting in UEFI mode. VESA module must be used when booting in BIOS mode.

This command can be used to check the booting mode:

```
% sysctl machdep.bootmethod
```

The output should be similar to the following:

```
machdep.bootmethod: UEFI
```

5.2.1. Intel® Graphics

The graphics/drm-kmod package indirectly provides a range of kernel modules for use with Intel® Graphics. Recent versions of these modules can be used in conjunction with other graphics processors in PRIME with no special configuration.

The Intel® Graphics driver can be installed by executing the following command:

```
# pkg install drm-kmod
```

Then add the module to /etc/rc.conf file, by executing the following command:

```
# sysrc kld_list+=i915kms
```

5.2.2. AMD® Graphics

The graphics/drm-kmod package indirectly provides kernel modules for a range of AMD® Graphics processors. The modules `amdgpu` or `radeonkms` can be used depending the generation of the hardware. The FreeBSD project maintains a AMD graphics support matrix showing support levels and to determine which driver must be used.

The AMD® Graphics drivers can be installed by executing the following command:

```
# pkg install drm-kmod
```

Enable the current module by adding it to the /etc/rc.conf file, executing the following command:

```
# sysrc kld_list+=amdgpu
```

For older graphics (pre-HD7000/Tahiti), instead enable the legacy module by adding it to the /etc/rc.conf file, executing the following command:

```
# sysrc kld_list+=radeonkms
```

5.2.3. NVIDIA® Graphics

NVIDIA® produces standalone or discrete graphics processors, and provides a proprietary driver for FreeBSD. The FreeBSD Ports Collection provides over a decade of drivers for supporting generations of NVIDIA graphics.

Administrators should install the latest driver supported by their hardware.

The following table shows the port containing the driver, the kernel module recommended for loading, and a link to the list of hardware supported by that driver:

Table 9. Supported versions of NVIDIA® Graphics drivers

Port	Module	Supported hardware
graphics/nvidia-drm-kmod	`nvidia-drm` or `nvidia-modeset`	supported hardware
x11/nvidia-driver-470	`nvidia-modeset`	supported hardware
x11/nvidia-driver-390 or x11/nvidia-secondary-driver-390	`nvidia-modeset`	supported hardware
x11/nvidia-driver-340	`nvidia`	supported hardware
x11/nvidia-driver-304	`nvidia`	supported hardware

The latest NVIDIA® Graphics driver can be installed by running the following command:

```
# pkg install nvidia-drm-kmod
```

To enable the driver, add the module to /etc/rc.conf file, by executing the following command:

```
# sysrc kld_list+=nvidia-drm
```

This is the direct rendering KMS driver.

Kernel modesetting is the option to set the graphics mode in the kernel. Enable it for subsequent boots with the following loader.conf(5) tunable:

```
hw.nvidiadrm.modeset="1"
```

Both PRIME and Wayland require kernel modesetting.

Prior versions of the driver do not support Direct Rendering. Instead use the modesetting module, by executing the following command:

```
# sysrc kld_list+=nvidia-modeset
```

If requiring Nvidia drivers prior to 390, note that they do not support kernel modesetting, and thus they must be used with the legacy sc(4) console driver, and a x11/xorg-server version prior to 1.20.

Enable them in /etc/rc.conf with the following command:

```
# sysrc kld_list+=nvidia
```

Some graphic cards will need special firmware that needs to be downloaded for the device to work properly. The fwget(8) utility can download and install the necessary files under /boot/firmware.

fwget(8) will scan the 'PCI' devices and download the firmware for those devices that need it.

5.3. X Window System Overview

The X Window System is the heritage graphical stack for UNIX® platforms, supporting the latest technologies while maintaining support for generations of applications. Applications, including the components of the desktop, are hosted by the Xorg(1) server. This system is network aware and its various components can interoperate across networks.

5.4. Installing The X.org Server

***abstract**: The X.org server must be installed to host the desktop. Users must be added to the `video` group to use it.*

Once a graphics driver is installed and enabled, the X.org server can be installed as a meta-package, or compiled locally with the ports tree.

The full meta-package can be installed quickly but with fewer options for customization:

```
# pkg install xorg
```

This installation results in the complete X Window System being installed, including a traditional window manager, twm(1), and surrounding traditional desktop suite. Most users will want to install and configure a contemporary desktop of their choice.

The current user must be a member of the `video` group to run a graphical environment. To add a user to the `video` group, execute the following command:

```
# pw groupmod video -m username
```

To run the X Window System, use startx(1) from x11/xinit, or install and configure a display manager to start a graphical login on boot.



A smaller version of the X Window System suitable for experienced users is available in x11/xorg-minimal. Most of the documents, libraries, and applications will not be installed. Some applications require these additional components to function.

5.5. X.org Configuration

***abstract**: If the defaults for the monitor or input devices, are not satisfactory, desktops include GUIs for configuring them, or they can be configured manually.*

The X.org server supports most common graphics processors, monitors, and input devices. First, try the defaults. This subsection provides an overview of their configuration.

5.5.1. X.org Configuration Files

Historically, the X.org server was configured with files in /usr/local/etc/X11/. This is still supported for edge cases, but conflicts with dynamic autoconfiguration.

Do not create configuration for the X.org server in xorg.conf or run `Xorg -configure` unless automatic configuration fails.

X.org server looks in several directories for configuration files. /usr/local/etc/X11/ is the **recommended** directory for these files on FreeBSD. Using this directory helps keep application files separate from operating system files.

It is easier to use multiple files that each configure a specific setting than the traditional single xorg.conf. These files are stored in the /usr/local/etc/X11/xorg.conf.d/ subdirectory.

5.5.2. Graphics Configuration

Direct rendering provides the ability to seamlessly use a discrete graphics processor (dGPU) alongside an integrated graphics processor (iGPU), called PRIME. The drivers will automatically offload intensive tasks to the dGPU when required, and power it down when able.

To launch applications on the more powerful GPU in PRIME, use the `DRI_PRIME=1` environment variable.

If multiple graphics drivers are conflicting, the driver for the graphics processor can be specified in the /usr/local/etc/X11/xorg.conf.d/ directory.

To configure the Intel® driver in a configuration file:

Example 14. Select Intel® Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-intel.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "intel"
EndSection
```

To configure the AMD® driver in a configuration file:

Example 15. Select AMD® Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-radeon.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "radeon"
EndSection
```

To configure the NVIDIA® driver in a configuration file:

Example 16. Select NVIDIA® Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-nvidia.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "nvidia"
EndSection
```



x11/nvidia-xconfig can also be used to perform basic control over configuration options available in the NVIDIA driver.

To configure the SCFB driver in a configuration file:

Example 17. Select SCFB Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-scfb.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "scfb"
EndSection
```

To configure the VESA driver in a configuration file:

Example 18. Select VESA Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-vesa.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "vesa"
EndSection
```

To configure multiple graphics processors, the `BusID` can be added. A list of graphics processor bus `ID`s can be displayed by executing:

```
% pciconf -lv | grep -B3 display
```

The output should be similar to the following:

```
vgapci0@pci0:0:2:0:     class=0x030000 rev=0x0c hdr=0x00 vendor=0x8086 device=0x46a6
subvendor=0x1028 subdevice=0x0b29
    vendor     = 'Intel Corporation'
    device     = 'Alder Lake-P GT2 [Iris Xe Graphics]'
    class      = display
--
vgapci0@pci0:1:0:0:     class=0x030200 rev=0xa1 hdr=0x00 vendor=0x10de device=0x25b9
subvendor=0x1028 subdevice=0x0b29
    vendor     = 'NVIDIA Corporation'
    device     = 'GA107GLM [RTX A1000 Laptop GPU]'
    class      = display
```

Example 19. Select Intel® Graphics Driver and NVIDIA® Graphics Driver in a File

/usr/local/etc/X11/xorg.conf.d/20-drivers.conf

```
Section "Device"
    Identifier "Card0"
    Driver     "intel"
    BusID     "pci0:0:2:0"
EndSection

Section "Device"
    Identifier "Card1"
    Driver     "nvidia"
    BusID     "pci0:1:0:0"
EndSection
```

5.5.3. Monitor Configuration

Almost all monitors support the Extended Display Identification Data standard (EDID). X.org uses EDID to communicate with the monitor and detect the supported resolutions and refresh rates. Then it selects the most appropriate combination of settings to use with that monitor.

Other resolutions supported by the monitor can be selected atomically after the X server has been started with xrandr(1), or in the X.org server configuration files.

5.5.3.1. Using RandR (Resize and Rotate)

Run xrandr(1) in an X session without any parameters to see a list of video outputs and detected monitor modes:

```
% xrandr
```

The output should be similar to the following:

```
Screen 0: minimum 320 x 200, current 2560 x 960, maximum 8192 x 8192
LVDS-1 connected 1280x800+0+0 (normal left inverted right x axis y axis) 261mm x 163mm
   1280x800      59.99*+  59.81    59.91    50.00
   1280x720      59.86    59.74
   1024x768      60.00
   1024x576      59.90    59.82
   960x540       59.63    59.82
   800x600       60.32    56.25
   864x486       59.92    59.57
   640x480       59.94
   720x405       59.51    58.99
   640x360       59.84    59.32
VGA-1 connected primary 1280x960+1280+0 (normal left inverted right x axis y axis)
410mm x 257mm
   1280x1024     75.02    60.02
   1440x900      74.98    60.07
   1280x960      60.00*
   1280x800      74.93    59.81
   1152x864      75.00
   1024x768      75.03    70.07    60.00
   832x624       74.55
   800x600       72.19    75.00    60.32    56.25
   640x480       75.00    72.81    66.67    59.94
   720x400       70.08
HDMI-1 disconnected (normal left inverted right x axis y axis)
DP-1 disconnected (normal left inverted right x axis y axis)
HDMI-2 disconnected (normal left inverted right x axis y axis)
DP-2 disconnected (normal left inverted right x axis y axis)
DP-3 disconnected (normal left inverted right x axis y axis)
```

This shows that the `VGA-1` output is being used to display a screen resolution of 1280x960 pixels at a refresh rate of about 60 Hz. The `LVDS-1` is being used as a secondary monitor to display a screen resolution of 1280x800 pixels at a refresh rate of about 60 Hz. Monitors are not attached to the `HDMI-1`, `HDMI-2`, `DP-1`, `DP-2`, and `DP-3` connectors.

Any of the other display modes can be selected with xrandr(1). For example, to switch to 1280x1024 at 60 Hz:

```
% xrandr --output LVDS-1 --mode 1280x720 --rate 60
```



Oftentimes, a black screen upon starting X can be fixed by adding an `xrandr --auto` step to the initialization process.

5.5.3.2. Using the X.org Configuration Files

The monitor configuration can also be set in a configuration file.

To set a screen resolution of 1024x768 in a configuration file:

Example 20. Set Screen Resolution in a File

/usr/local/etc/X11/xorg.conf.d/10-monitor.conf

```
Section "Screen"
    Identifier "Screen0"
    Device     "Card0"
    SubSection "Display"
    Modes      "1024x768"
    EndSubSection
EndSection
```

5.5.4. Input Configuration

The X.org server provides the x11/libinput library, a cross-platform effort to support all touch, pointing, and keyboard devices under a unified library. Unless specified otherwise, this is loaded automatically.

Individual device settings for libinput(4) can be tuned in the desktop's GUI, or manually with xinput and setxkbmap.

Alternatively, there are older, lightweight, individual drivers for specific input devices available in the x11-drivers category named x11/xf86-input-[foo]. This approach requires manual configuration of the X.org server. Both are described in this subsection.

5.5.4.1. Using Atomic Input Configuration

Devices supported by libinput(4) can be configured with graphical utilities included with the desktop of choice, or manually and atomically at runtime with x11/xinput and x11/setxkbmap.

To ask libinput(4) what devices it's currently attached to, run xinput(1) with no arguments:

```
$ xinput
```

Its output should be similar to the following:

```
⎡ Virtual core pointer                    id=2    [master pointer  (3)]
⎜   ↳ Virtual core XTEST pointer              id=4    [slave  pointer  (2)]
⎜   ↳ System mouse                            id=7    [slave  pointer  (2)]
⎜   ↳ VEN_0488:00 0488:1031 Mouse             id=11   [slave  pointer  (2)]
⎜   ↳ VEN_0488:00 0488:1031 TouchPad          id=12   [slave  pointer  (2)]
⎣ Virtual core keyboard                   id=3    [master keyboard (2)]
    ↳ Virtual core XTEST keyboard             id=5    [slave  keyboard (3)]
    ↳ System keyboard multiplexer             id=6    [slave  keyboard (3)]
    ↳ Power Button                            id=8    [slave  keyboard (3)]
    ↳ Sleep Button                            id=9    [slave  keyboard (3)]
```

```
    ⎜   AT keyboard                          id=10   [slave  keyboard (3)]
```

All settings supported by these devices are provided as properties, which can be listed and set atomically. Pointing devices have many configurable properties, keyboards usually need none.

To customize the keyboard, take a look at setxkbmap(1).

Once satisfied with the configuration, simply add the lines to the X initialization script such as ~/.Xsession or ~/.xinitrc.

5.5.4.2. Using X.org Configuration Files



Some desktop environments (such as KDE Plasma) provide a graphical UI for setting these parameters. Check if this is the case before resorting to manual configuration editing.

For example, to manually configure the X.org server for the keyboard layout:

Example 21. Setting a Keyboard Layout

/usr/local/etc/X11/xorg.conf.d/00-keyboard.conf

```
Section "InputClass"
        Identifier "Keyboard1"
        MatchIsKeyboard "on"
        Option "XkbLayout" "es, fr"
        Option "XkbModel" "pc104"
        Option "XkbVariant" ",qwerty"
        Option "XkbOptions" "grp:win_space_toggle"
EndSection
```

5.6. Using Fonts in the X Window System

***abstract**: Additional fonts can be installed from the x11-fonts category or placed in ~/.fonts. They are available immediately to modern applications. Configuration for older applications is available and described as well.*

The X Window System provides the X FreeType interface library (Xft(3)) to render vector or outline fonts, as well as the traditional X Logical Font Description system maintaining compatibility with generations of applications and fonts.

There are primarily two types of fonts users will be interested in:

- OpenType fonts or TrueType® fonts are for displaying on a screen.
- Adobe® PostScript® Type 1 fonts are for printing to paper.

These are both vector or outline fonts, there are also bitmap fonts.

The FreeBSD Ports Collection includes a wide and growing catalog of free, high quality fonts available for installation in the x11-fonts category.

System-wide font packages installed from the ports collection live in `/usr/local/share/fonts/`. Fonts for a single user can be placed in `~/.fonts/`, or `~/.local/share/fonts/`.

Fonts in either directory or subdirectories will be available for immediate use when the font information cache is rebuilt. To trigger this manually, issue:

```
% fc-cache
```

Plenty of free, high quality fonts of both types are available in the ports tree which can be readily used with X Window System. This chapter provides a brief overview of both, as well as configuring the X FreeType interface.

For more information about how to install and configure fonts on FreeBSD, please read the article Fonts and FreeBSD.

5.6.1. TrueType® Fonts

X.org has built in support for rendering TrueType® fonts. There are two different modules that can enable this functionality. The freetype module is used in this example because it is more consistent with the other font rendering back-ends. To enable the freetype module just add the following line to the `"Module"` section of /usr/local/etc/X11/xorg.conf.d/90-fonts.conf.

```
Load  "freetype"
```

Now make a directory for the TrueType® fonts (for example, /usr/local/share/fonts/TrueType) and copy all of the TrueType® fonts into this directory. Keep in mind that TrueType® fonts cannot be directly taken from an Apple® Mac®; they must be in UNIX®/MS-DOS®/Windows® format for use by X.org. Once the files have been copied into this directory, use mkfontscale to create a fonts.dir, so that the X font renderer knows that these new files have been installed. `mkfontscale` can be installed as a package:

```
# pkg install mkfontscale
```

Then create an index of X font files in a directory:

```
# cd /usr/local/share/fonts/TrueType
# mkfontscale
```

Now add the TrueType® directory to the font path. This is just the same as described in Type1 Fonts:

```
% xset fp+ /usr/local/share/fonts/TrueType
```

```
% xset fp rehash
```

or add a `FontPath` line to xorg.conf.

Now Gimp, LibreOffice, and all of the other X applications should now recognize the installed TrueType® fonts. Extremely small fonts (as with text in a high resolution display on a web page) and extremely large fonts (within LibreOffice) will look much better now.

5.6.2. Type1 Fonts

The URW font collection (x11-fonts/urwfonts) includes high quality versions of standard type1 fonts (Times Roman™, Helvetica™, Palatino™ and others). The Freefonts collection (x11-fonts/freefonts) includes many more fonts, but most of them are intended for use in graphics software such as the Gimp, and are not complete enough to serve as screen fonts.

To install the above Type1 font collections from binary packages, run the following commands:

```
# pkg install urwfonts
```

And likewise with the freefont or other collections. To have a manually configured X server detect these fonts, add an appropriate line to the X server configuration file (/usr/local/etc/X11/xorg.conf.d/90-fonts.conf), which reads:

```
Section "Files"
  FontPath "/usr/local/share/fonts/urwfonts/"
EndSection
```

Alternatively, at the command line in the X session run:

```
% xset fp+ /usr/local/share/fonts/urwfonts
% xset fp rehash
```

This will work but will be lost when the X session is closed, unless it is added to the startup file (~/.xinitrc for a normal `startx` session, or ~/.xsession when logging in through a graphical login manager like XDM). A third way is to use the new /usr/local/etc/fonts/local.conf as demonstrated in Anti-Aliased Fonts.

5.6.3. Anti-Aliased Fonts

All fonts in X.org that are found in /usr/local/share/fonts/ and ~/.fonts/ are automatically made available for anti-aliasing to Xft-aware applications. Most recent applications are Xft-aware, including KDE, GNOME, and Firefox.

To control which fonts are anti-aliased, or to configure anti-aliasing properties, create (or edit, if it already exists) the file /usr/local/etc/fonts/local.conf. Several advanced features of the Xft font system can be tuned using this file; this section describes only some simple possibilities. For more

details, please see fonts-conf(5).

This file must be in XML format. Pay careful attention to case, and make sure all tags are properly closed. The file begins with the usual XML header followed by a DOCTYPE definition, and then the `<fontconfig>` tag:

```
<?xml version="1.0"?>
      <!DOCTYPE fontconfig SYSTEM "fonts.dtd">
      <fontconfig>
```

As previously stated, all fonts in /usr/local/share/fonts/ as well as ~/.fonts/ are already made available to Xft-aware applications. To add another directory outside of these two directory trees, add a line like this to /usr/local/etc/fonts/local.conf:

```
<dir>/path/to/my/fonts</dir>
```

After adding new fonts, and especially new font directories, rebuild the font caches:

```
# fc-cache -f
```

Anti-aliasing makes borders slightly fuzzy, which makes very small text more readable and removes "staircases" from large text, but can cause eyestrain if applied to normal text. To exclude font sizes smaller than 14 point from anti-aliasing, include these lines:

```
    <match target="font">
        <test name="size" compare="less">
        <double>14</double>
        </test>
        <edit name="antialias" mode="assign">
        <bool>false</bool>
        </edit>
    </match>
    <match target="font">
        <test name="pixelsize" compare="less" qual="any">
        <double>14</double>
        </test>
        <edit mode="assign" name="antialias">
        <bool>false</bool>
        </edit>
    </match>
```

Spacing for some monospaced fonts might also be inappropriate with anti-aliasing. This seems to be an issue with KDE, in particular. One possible fix is to force the spacing for such fonts to be 100. Add these lines:

```
<match target="pattern" name="family">
   <test qual="any" name="family">
       <string>fixed</string>
   </test>
   <edit name="family" mode="assign">
       <string>mono</string>
   </edit>
</match>
<match target="pattern" name="family">
    <test qual="any" name="family">
    <string>console</string>
    </test>
    <edit name="family" mode="assign">
    <string>mono</string>
    </edit>
</match>
```

(this aliases the other common names for fixed fonts as "mono"), and then add:

```
<match target="pattern" name="family">
     <test qual="any" name="family">
     <string>mono</string>
     </test>
     <edit name="spacing" mode="assign">
     <int>100</int>
     </edit>
 </match>
```

Certain fonts, such as Helvetica, may have a problem when anti-aliased. Usually this manifests itself as a font that seems cut in half vertically. At worst, it may cause applications to crash. To avoid this, consider adding the following to local.conf:

```
<match target="pattern" name="family">
     <test qual="any" name="family">
     <string>Helvetica</string>
     </test>
     <edit name="family" mode="assign">
     <string>sans-serif</string>
     </edit>
 </match>
```

After editing local.conf, make certain to end the file with the </fontconfig> tag. Not doing this will cause changes to be ignored.

Users can add personalized settings by creating their own ~/.config/fontconfig/fonts.conf. This file uses the same XML format described above.

One last point: with an LCD screen, sub-pixel sampling may be desired. This basically treats the (horizontally separated) red, green and blue components separately to improve the horizontal resolution; the results can be dramatic. To enable this, add the line somewhere in local.conf:

```
<match target="font">
    <test qual="all" name="rgba">
    <const>unknown</const>
    </test>
    <edit name="rgba" mode="assign">
    <const>rgb</const>
    </edit>
</match>
```



Depending on the sort of display, `rgb` may need to be changed to `bgr`, `vrgb` or `vbgr`: experiment and see which works best.

Chapter 6. Wayland on FreeBSD

6.1. Synopsis

An installation of FreeBSD using bsdinstall does not automatically install a graphical user interface. This chapter describes how to select, install, and configure a Wayland compositor, which provides a graphical environment.

Before reading this chapter:

- Know how to install additional third-party software.
- Know how to identify and configure drivers for the graphics hardware.

Read this chapter to learn:

- How to configure FreeBSD to host a Wayland graphical environment.
- How to install and configure a Wayland compositor.
- How to run programs designed for the older X Window System.
- How to configure remote desktop access to a Wayland graphical environment.

6.2. Wayland Overview

Wayland is a communication protocol that can replace a display server such as Xorg. It differs from Xorg in several important ways. First, Wayland is only a protocol that acts as an intermediary between clients using a mechanism which removes the dependency on an X server. Xorg includes both the X11 protocol, used to run remote displays, and the X server, used to accept connections and display windows. Under Wayland, the compositor or window manager provides the display server instead of a traditional X server.

Since Wayland is not an X server, traditional X screen connections will need to utilize other methods such as VNC or RDP for remote desktop management. Second, Wayland can manage composite communications between clients and a compositor as a separate entity which does not need to support the X protocols.

Wayland is relatively new, and not all software has been updated to run natively without `Xwayland` support. Because Wayland does not provide the X server, and expects compositors to provide that support, X11 window managers that do not yet support Wayland will require that `Xwayland` is not started with the `-rootless` parameter. The `-rootless` parameter, when removed, will restore X11 window manager support.



The current NVIDIA® driver should work with most wlroots compositors, but it may be a little unstable and not support all features at this time. Volunteers to help work on the NVIDIA® DRM are requested.

Currently, a lot of software will function with minimal issues on Wayland, including Firefox. And a few desktops are also available, such as the Compiz Fusion replacement, known as Wayfire, and the

i3 window manager replacement, Sway.



As of May, 2021, plasma5-kwin does support Wayland on FreeBSD. To use Plasma under Wayland, use the `startplasma-wayland` parameter to `ck-launch-session` and tie in dbus with: `dbus-launch --exit-with-x11 ck-launch-session startplasma-wayland` to get it working.

For compositors, a kernel supporting the evdev(4) driver must exist to utilize the keybinding functionality. This is built into the GENERIC kernel by default; however, if it has been customized and evdev(4) support was stripped out, the evdev(4) module will need to be loaded. In addition, users of `Wayland` will need to be members of the `video` group. To quickly make this change, use the `pw` command:

```
pw groupmod video -m user
```

Installing Wayland is simple; there is not a great deal of configuration for the protocol itself. Most of the composition will depend on the chosen compositor. By installing `seatd` now, a step is skipped as part of the compositor installation and configuration as `seatd` is needed to provide non-root access to certain devices.

All of the compositors described here should work with graphics/drm-kmod open source drivers; however, the NVIDIA® graphics cards may have issues when using the proprietary drivers. Begin by installing the following packages:

```
# pkg install wayland seatd
```

Once the protocol and supporting packages have been installed, a compositor must create the user interface. Several compositors will be covered in the following sections. All compositors using Wayland will need a runtime directory defined in the environment. Since FreeBSD 14.1, this is created and defined automatically. For previous versions, this is achieved with the following command in the bourne shell:

```
% export XDG_RUNTIME_DIR=/var/run/user/`id -u`
```

It is important to note that most compositors will search the XDG_RUNTIME_DIR directory for the configuration files. In the examples included here, a parameter will be used to specify a configuration file in ~/.config to keep temporary files and configuration files separate. It is recommended that an alias be configured for each compositor to load the designated configuration file.



It has been reported that ZFS users may experience issues with some Wayland clients because they need access to `posix_fallocate()` in the runtime directory. While the author could not reproduce this issue on their ZFS system, a recommended workaround is not to use ZFS for the runtime directory and instead use `tmpfs` for the /var/run directory. In this case, the `tmpfs` file system is used for

> /var/run and mounted through the command `mount -t tmpfs tmpfs /var/run` command and then make this change persist across reboots through /etc/fstab. The XDG_RUNTIME_DIR environment variable could be configured to use /var/run/user/$UID and avoid potential pitfalls with ZFS. Consider that scenario when reviewing the configuration examples in the following sections.

The seatd daemon helps manage access to shared system devices for non-root users in compositors; this includes graphics cards. For traditional X11 managers, `seatd` is not needed, such as both Plasma and GNOME, but for the Wayland compositors discussed here, it will need enabled on the system and be running before starting a compositor environment. To enable and start the `seatd` daemon now, and on system initialization:

```
# sysrc seatd_enable="YES"
# service seatd start
```

Afterward, a compositor, which is similar to an X11 desktop, will need to be installed for the GUI environment. Three are discussed here, including basic configuration options, setting up a screen lock, and recommendations for more information.

6.3. The Wayfire Compositor

Wayfire is a compositor that aims to be lightweight and customizable. Several features are available, and it brings back several elements from the previously released Compiz Fusion desktop. All of the parts look beautiful on modern hardware. To get Wayfire up and running, begin by installing the required packages:

```
# pkg install wayfire wf-shell alacritty swaylock-effects swayidle wlogout kanshi mako
wlsunset
```

The `alacritty` package provides a terminal emulator. Still, it is not completely required as other terminal emulators such as `kitty`, and XFCE-4 `Terminal` have been tested and verified to work under the Wayfire compositor. Wayfire configuration is relatively simple; it uses a file that should be reviewed for any customizations. To begin, copy the example file over to the runtime environment configuration directory and then edit the file:

```
% mkdir ~/.config/wayfire
% cp /usr/local/share/examples/wayfire/wayfire.ini ~/.config/wayfire
```

The defaults for most users should be fine. Within the configuration file, items like the famous `cube` are pre-configured, and there are instructions to help with the available settings. A few primary settings of note include:

```
[output]
mode = 1920x1080@60000
position = 0,0
```

```
transform = normal
scale = 1.000000
```

In this example, from the configuration file, the screen's output should be the listed mode at the listed hertz. For example, the mode should be set to `widthxheight@refresh_rate`. The position places the output at a specific pixel location specified. The default should be fine for most users. Finally, transform sets a background transform, and scale will scale the output to the specified scale factor. The defaults for these options are generally acceptable; for more information, see the documentation.

As mentioned, Wayland is new, and not all applications work with the protocol yet. At this time, `sddm` does not appear to support starting and managing compositors in Wayland. The `swaylock` utility has been used instead in these examples. The configuration file contains options to run `swayidle` and `swaylock` for idle and locking of the screen.

This option to define the action to take when the system is idle is listed as:

```
idle = swaylock
```

And the lock timeout is configured using the following lines:

```
[idle]
toggle = <super> KEY_Z
screensaver_timeout = 300
dpms_timeout = 600
```

The first option will lock the screen after 300 seconds, and after another 300, the screen will shut off through the `dpms_timeout` option.

One final thing to note is the <super> key. Most of the configuration mentions this key, and it is the traditional `Windows` key on the keyboard. Most keyboards have this super key available; however, it should be remapped within this configuration file if it is not available. For example, to lock the screen, press and hold the super key, the `shift` key, and press the `escape` key. Unless the mappings have changed, this will execute the swaylock application. The default configuration for `swaylock` will show a grey screen; however, the application is highly customizable and well documented. In addition, since the swaylock-effects is the version that was installed, there are several options available such as the blur effect, which can be seen using the following command:

```
% swaylock --effect-blur 7x5
```

There is also the `--clock` parameter which will display a clock with the date and time on the lock screen. When x11/swaylock-effects was installed, a default pam.d configuration was included. It provides the default options that should be fine for most users. More advanced options are available; see the PAM documentation for more information.

At this point, it is time to test Wayfire and see if it can start up on the system. Just type the following

command:

```
% wayfire -c ~/.config/wayfire/wayfire.ini
```

The compositor should now start and display a background image along with a menu bar at the top of the screen. Wayfire will attempt to list installed compatible applications for the desktop and present them in this drop-down menu; for example, if the XFCE-4 file manager is installed, it will show up in this drop-down menu. If a specific application is compatible and valuable enough for a keyboard shortcut, it may be mapped to a keyboard sequence using the wayfire.ini configuration file. Wayfire also has a configuration tool named Wayfire Config Manager. It is located in the drop-down menu bar but may also be started through a terminal by issuing the following command:

```
% wcm
```

Various Wayfire configuration options, including the composite special effects, maybe enabled, disabled, or configured through this application. In addition, for a more user-friendly experience, a background manager, panel, and docking application may be enabled in the configuration file:

```
panel = wf-panel
dock = wf-dock
background = wf-background
```



Changes made through `wcm` will overwrite custom changes in the wayfire.ini configuration file. The wayfire.ini file is highly recommended to be backed up so any essential changes may be restored.

Finally, the default launcher listed in the wayfire.ini is x11/wf-shell which may be replaced with other panels if desired by the user.

6.4. The Hikari Compositor

The Hikari compositor uses several concepts centered around productivity, such as sheets, workspaces, and more. In that way, it resembles a tiling window manager. Breaking this down, the compositor starts with a single workspace, which is similar to virtual desktops. Hikari uses a single workspace or virtual desktop for user interaction. The workspace is made up of several views, which are the working windows in the compositor grouped as either sheets or groups. Both sheets and groups are made up of a collection of views; again, the windows that are grouped together. When switching between sheets or groups, the active sheet or group will become known collectively as the workspace. The manual page will break this down into more information on the functions of each but for this document, just consider a single workspace utilizing a single sheet. Hikari installation will comprise of a single package, x11-wm/hikari, and a terminal emulator `alacritty`:

```
# pkg install hikari alacritty
```



Other shells, such as `kitty` or the Plasma `Terminal`, will function under Wayland. Users should experiment with their favorite terminal editor to validate compatibility.

Hikari uses a configuration file, hikari.conf, which could either be placed in the XDG_RUNTIME_DIR or specified on startup using the `-c` parameter. An autostart configuration file is not required but may make the migration to this compositor a little easier. Beginning the configuration is to create the Hikari configuration directory and copy over the configuration file for editing:

```
% mkdir ~/.config/hikari
% cp /usr/local/etc/hikari/hikari.conf ~/.config/hikari
```

The configuration is broken out into various stanzas such as ui, outputs, layouts, and more. For most users, the defaults will function fine; however, some important changes should be made. For example, the $TERMINAL variable is normally not set within the user's environment. Changing this variable or altering the hikari.conf file to read:

```
terminal = "/usr/local/bin/alacritty"
```

Will launch the `alacritty` terminal using the bound key press. While going through the configuration file, it should be noted that the capital letters are used to map keys out for the user. For example, the `L` key for starting the terminal `L` + `Return` is actually the previously discussed super key or Windows logo key. Therefore, holding the `L/super/Windows` key and pressing `Enter` will open the specified terminal emulator with the default configuration. Mapping other keys to applications require an action definition to be created. For this, the action item should be listed in the actions stanza, for example:

```
actions {
  terminal = "/usr/local/bin/alacritty"
  browser = "/usr/local/bin/firefox"
}
```

Then an action may be mapped under the keyboard stanza, which is defined within the bindings stanza:

```
bindings {
  keyboard {
SNIP
    "L+Return" = action-terminal
    "L+b" = action-browser
SNIP
```

After Hikari is restarted, holding the Windows logo button and pressing the `b` key on the keyboard will start the web browser. The compositor does not have a menu bar, and it is recommended the

user set up, at minimal, a terminal emulator before migration. The manual page contains a great deal of documentation it should be read before performing a full migration. Another positive aspect about Hikari is that, while migrating to the compositor, Hikari can be started in the Plasma and GNOME desktop environments, allowing for a test-drive before completely migrating.

Locking the screen in Hikari is easy because a default pam.d configuration file and unlock utility are bundled with the package. The key binding for locking the screen is `L` (Windows logo key)+ `Shift` + `Backspace`. It should be noted that all views not marked public will be hidden. These views will never accept input when locked but beware of sensitive information being visible. For some users, it may be easier to migrate to a different screen locking utility such as swaylock-effects, discussed in this section. To start Hikari, use the following command:

```
% hikari -c ~/.config/hikari/hikari.conf
```

6.5. The Sway Compositor

The Sway compositor is a tiling compositor that attempts to replace the i3 windows manager. It should work with the user's current i3 configuration; however, new features may require some additional setup.

Before starting the Sway installation, ensure that a graphics card (GPU) is installed and configured correctly. Refer to the section drivers for the graphics hardware for guidance. This step is essential for the Sway compositor to function properly.

In the forthcoming examples, a fresh installation without migrating any i3 configuration will be assumed. To install Sway and valuable components, issue the following command as the root user:

```
# pkg install sway swayidle swaylock-effects alacritty dmenu-wayland dmenu
```

For a basic configuration file, issue the following commands and then edit the configuration file after it is copied:

```
% mkdir ~/.config/sway
% cp /usr/local/etc/sway/config ~/.config/sway
```

The base configuration file has many defaults, which will be fine for most users. Several important changes should be made like the following:

```
# Logo key. Use Mod1 for Alt.
input * xkb_rules evdev
set $mod Mod4
# The preferred terminal emulator
set $term alacritty
set $lock swaylock -f -c 000000
output "My Workstation" mode 1366x768@60Hz position 1366 0
```

```
output * bg ~/wallpapers/mywallpaper.png stretch
### Idle configuration
exec swayidle -w \
          timeout 300 'swaylock -f -c 000000' \
          timeout 600 'swaymsg "output * dpms off"' resume 'swaymsg "output * dpms
on"' \
          before-sleep 'swaylock -f -c 000000'
```

In the previous example, the `xkb` rules for evdev(4) events are loaded, and the $mod key is set to the Windows logo key for the key bindings. Next, the terminal emulator was set to be `alacritty`, and a screen lock command was defined; more on this later. The output keyword, the mode, the position, a background wallpaper, and Sway is also told to stretch this wallpaper to fill out the screen. Finally, `swayidle` is set to daemonize and lock the screen after a timeout of 300 seconds, placing the screen or monitor into sleep mode after 600 seconds. The locked background color of 000000, which is black, is also defined here. Using swaylock-effects, a clock may also be displayed with the `--clock` parameter. See the manual page for more options. The sway-output(5) manual page should also be reviewed; it includes a great deal of information on customing the output options available.

While in Sway, to bring up a menu of applications, hold the Windows logo key (mod) and press the `d` key. The menu may be navigated using the arrow keys on the keyboard. There is also a method to manipulate the layout of the bar and add a tray; read the sway-bar(5) manual page for more information. The default configuration adds a date and time to the upper right-hand corner. See the `Bar` stanza in the configuration file for an example. By default, the configuration does not include locking the screen outside of the example above, enabling a lockout timer. Creating a lock key binding requires the following line to the `Key bindings` section:

```
# Lock the screen manually
bindsym $mod+Shift+Return exec $lock
```

Now the screen may be locked using the combination of holding the Windows logo key, pressing and holding shift, and finally pressing return. When Sway is installed, whether from a package or the FreeBSD Ports Collection, a default file for pam.d was installed. The default configuration should be acceptable for most users, but more advanced options are available. Read through the PAM documentation for more information.

Finally, to exit Sway and return to the shell, hold the Windows logo key, the shift key, and press the `e` key. A prompt will be displayed with an option to exit Sway. During migration, Sway can be started through a terminal emulator on an X11 desktop such as Plasma. This makes testing different changes and key bindings a little easier prior to fully migrating to this compositor. To start Sway, issue the following command:

```
% sway -c ~/.config/sway/config
```

6.6. Using Xwayland

When installing Wayland, the `Xwayland` binary should have been installed unless Wayland was built

without X11 support. If the /usr/local/bin/Xwayland file does not exist, install it using the following command:

```
# pkg install xwayland
```



The development version of Xwayland is recommended and was most likely installed with the Wayland package. Each compositor has a method of enabling or disabling this feature.

Once `Xwayland` has been installed, configure it within the chosen compositor. For Wayfire, the following line is required in the wayfire.ini file:

```
xwayland = true
```

For the Sway compositor, `Xwayland` should be enabled by default. Even so, it is recommended to manually add a configuration line in the ~/.config/sway/config like the following:

```
xwayland enable
```

Finally, for Hikari, no changes are needed. Support for `Xwayland` is built in by default. To disable that support, rebuild the package from the ports collection and disable Xwayland support at that time.

After these changes are made, start the compositor at the command line and execute a terminal from the key bindings. Within this terminal, issue the `env` command and search for the `DISPLAY` variables. If the compositor was able to properly start the Xwayland X server, these environment variables should look similar to the following:

```
% env | grep DISPLAY
```

```
WAYLAND_DISPLAY=wayland-1
DISPLAY=:0
```

In this output, there is a default Wayland display and a display set for the Xwayland server. Another method to verify that `Xwayland` is functioning properly is to use install and test the small package:[x11/eyes] and check the output. If the `xeyes` application starts and the eyes follow the mouse pointer, Xwayland is functioning properly. If an error such as the following is displayed, something happened during the `Xwayland` initialization and it may need reinstalled:

```
Error: Cannot open display wayland-0
```



A security feature of Wayland is that, without running an X server, there is not another network listener. Once `Xwayland` is enabled, this security feature is no

> longer applicable to the system.

For some compositors, such as Wayfire, `Xwayland` may not start properly. As such, `env` will show the following information for the `DISPLAY` environment variables:

```
% env | grep DISPLAY
```

```
DISPLAY=wayland-1
WAYLAND_DISPLAY=wayland-1
```

Even though `Xwayfire` was installed and configured, X11 applications will not start giving a display issue. To work around this, verify that there is already an instance of `Xwayland` using a UNIX socket through these two methods. First, check the output from `sockstat` and search for X11-unix:

```
% sockstat | grep x11
```

There should be something similar to the following information:

```
trhodes  Xwayland   2734  8  stream /tmp/.X11-unix/X0
trhodes  Xwayland   2734  9  stream /tmp/.X11-unix/X0
trhodes  Xwayland   2734  10 stream /tmp/.X11-unix/X0
trhodes  Xwayland   2734  27 stream /tmp/.X11-unix/X0_
trhodes  Xwayland   2734  28 stream /tmp/.X11-unix/X0
```

This suggests the existence of an X11 socket. This can be further verified by attempting to execute `Xwayland` manually within a terminal emulator running under the compositor:

```
% Xwayland
```

If an X11 socket is already available, the following error should be presented to the user:

```
(EE)
Fatal server error:
(EE) Server is already active for display 0
    If this server is no longer running, remove /tmp/.X0-lock
    and start again.
(EE)
```

Since there is an active X display available using display zero, the environment variable was just set improperly, to fix this, change the `DISPLAY` environment variable to `:0` and attempt to execute the application again. The following example uses mail/claws-mail as the application which needs the `Xwayland` service:

```
export DISPLAY=:0
```

After this change, the mail/claws-mail application should now start using Xwayland and function as expected.

6.7. Remote Desktop Using VNC

Earlier in this document it was noted that Wayland does not provide the same X server style access as Xorg provides. Instead, users are free to pick and choose a remote desktop protocol such as RDP or VNC. The FreeBSD Ports collection includes the wayvnc, which will support wlroots based compositors such as the ones discussed here. This application may be installed using:

```
# pkg install wayvnc
```

Unlike some other packages, wayvnc does not come with a configuration file. Thankfully, the manual page documents the important options and they may be extrapolated into a simple configuration file:

```
address=0.0.0.0
enable_auth=true
username=username
password=password
private_key_file=/path/to/key.pem
certificate_file=/path/to/cert.pem
```

The key files will need to be generated, and it is highly recommended they be used for increased security of the connection. When invoked, wayvnc will search for the configuration file in ~/.config/wayvnc/config. This could be overwritten using the `-C configuration_file` option when starting the server. Thus, to start the wayvnc server, issue the following command:

```
% wayvnc -C ~/.config/wayvnc/config
```



At the time of this writing, there is no rc.d script to start wayvnc on system initialization. If that functionality is desired, a local startup file will need to be created. This is probably a feature request for the port maintainer.

6.8. Wayland Login Manager

While several login managers exist and are slowly migrating to Wayland, one option is the x11/ly text user interface (TUI) manager. Needing minimal configuration, ly will start Sway, Wayfire, and others by presenting a login window on system initialization. To install ly, issue the following command:

```
# pkg install ly
```

There will be some configuration hints presented, the import steps are to add the following lines to /etc/gettytab:

```
Ly:\
  :lo=/usr/local/bin/ly:\
  :al=root:
```

And then modify the ttyv1 line in /etc/ttys to match the following line:

```
ttyv1 "/usr/libexec/getty Ly" xterm onifexists secure
```

After a system reboot, a login should appear. To configure specific settings, such as language and edit /usr/local/etc/ly/config.ini. At minimal, this file should have the designated tty that was previously specified in /etc/ttys.



If setting ttyv0 up as the login terminal, it may be required to press the `alt` and `F1` keys to properly see the login window.

When the login window appears, using the left and right arrows will swap through different, supported, window managers.

6.9. Useful Utilities

One useful Wayland utility which all compositors can make use of is the waybar. While Wayfire does come with a launch menu, an easy-to-use and fast taskbar is a good accessory for any compositor or desktop manager. A Wayland compatible taskbar that is fast and easy to configure is waybar. To install the package and a supporting audio control utility, issue the following command:

```
# pkg install pavucontrol waybar
```

To create the configuration directory and copy over a default configuration file, execute the following commands:

```
% mkdir ~/.config/waybar
% cp /usr/local/etc/xdg/waybar/config ~/.config/waybar
```

The `lavalauncher` utility provides a launch bar for various applications. There is no example configuration file provided with the package, so the following actions must be taken:

```
mkdir ~/.config/lavalauncher
```

An example configuration file that only includes Firefox, and is placed on the right, is below:

```
global-settings {
    watch-config-file = true;
}

bar {
    output            = eDP-1;
    position          = bottom;
    background-colour = "#202020";

    # Condition for the default configuration set.
    condition-resolution = wider-than-high;

    config {
        position = right;
    }

    button {
        image-path          =
/usr/local/lib/firefox/browser/chrome/icons/default/default48.png;
        command[mouse-left] =     /usr/local/bin/firefox;
    }
    button {
      image-path           =   /usr/local/share/pixmaps/thunderbird.png;
      command[mouse-left]  =   /usr/local/bin/thunderbird;
}
```

Chapter 7. Network

7.1. Synopsis

This chapter delves into the topic of network configuration and performance, showcasing the robust networking capabilities of the FreeBSD operating system. Whether working with wired or wireless networks, this chapter provides a comprehensive guide to configuring and optimizing network connectivity in FreeBSD.

Before diving into the details, it is beneficial for readers to have a basic understanding of networking concepts such as protocols, network interfaces, and addressing.

This chapter covers:

- The ability to configure wired networks in FreeBSD, including network interface setup, addressing, and customization options.
- The skills to configure wireless networks in FreeBSD, encompassing wireless network interface setup, security protocols, and troubleshooting techniques.
- FreeBSD's networking capabilities and its reputation for excellent network performance.
- An understanding of various network services and protocols supported by FreeBSD, with configuration instructions for DNS, DHCP and more.

More information about how to make advanced network configurations in Advanced Networking.

7.2. Setting up the Network

Setting up a wired or wireless connection is a common task for a FreeBSD user. This section will show how to identify the wired and wireless network adapters and how to configure them.

Before starting with the configuration it is necessary to know the following network data:

- If the network has DHCP
- If the network does not have DHCP, the static IP to be used
- The netmask
- The IP address of the default gateway



The network connection may have been configured at installation time by bsdinstall(8).

7.2.1. Identify Network Adapters

FreeBSD supports a wide variety of network adapters for both wired and wireless networks. Check the Hardware Compatibility List for the used FreeBSD release to see if the network adapter is supported.

To get the network adapters used by our system execute the following command:

```
% pciconf -lv | grep -A1 -B3 network
```

The output should be similar to the following:

```
em0@pci0:0:25:0:        class=0x020000 rev=0x03 hdr=0x00 vendor=0x8086 device=0x10f5
subvendor=0x17aa subdevice=0x20ee
    vendor     = 'Intel Corporation' ①
    device     = '82567LM Gigabit Network Connection' ②
    class      = network
    subclass   = ethernet
--
iwm0@pci0:3:0:0:        class=0x028000 rev=0x00 hdr=0x00 vendor=0x8086 device=0x4237
subvendor=0x8086 subdevice=0x1211
    vendor     = 'Intel Corporation' ①
    device     = 'PRO/Wireless 5100 AGN [Shiloh] Network Connection' ②
    class      = network
```

The text before the '@' symbol is the name of the driver controlling the device. In this case these are em(4) and iwn(4).

① Shows the name of the vendor

② Shows the name of the device



It is only necessary to load the network interface card module if FreeBSD has not detected it correctly.

For example, to load the alc(4) module, execute the following command:

```
# kldload if_alc
```

Alternatively, to load the driver as a module at boot time, place the following line in /boot/loader.conf:

```
if_alc_load="YES"
```

7.3. Wired Networks

Once the right driver is loaded the network adapter needs to be configured. FreeBSD uses the driver name followed by a unit number to name the network interface adapter. The unit number represents the order in which the adapter is detected at boot time, or is later discovered.

For example, `em0` is the first network interface card (NIC) on the system using the em(4) driver.

To display the network interface configuration, enter the following command:

```
% ifconfig
```

The output should be similar to the following:

```
em0: flags=8863<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500

options=481249b<RXCSUM,TXCSUM,VLAN_MTU,VLAN_HWTAGGING,VLAN_HWCSUM,LRO,WOL_MAGIC,VLAN_H
WFILTER,NOMAP>
        ether 00:1f:16:0f:27:5a
        inet6 fe80::21f:16ff:fe0f:275a%em0 prefixlen 64 scopeid 0x1
        inet 192.168.1.19 netmask 0xffffff00 broadcast 192.168.1.255
        media: Ethernet autoselect (1000baseT <full-duplex>)
        status: active
        nd6 options=23<PERFORMNUD,ACCEPT_RTADV,AUTO_LINKLOCAL>
lo0: flags=8049<UP,LOOPBACK,RUNNING,MULTICAST> metric 0 mtu 16384
        options=680003<RXCSUM,TXCSUM,LINKSTATE,RXCSUM_IPV6,TXCSUM_IPV6>
        inet6 ::1 prefixlen 128
        inet6 fe80::1%lo0 prefixlen 64 scopeid 0x2
        inet 127.0.0.1 netmask 0xff000000
        groups: lo
        nd6 options=21<PERFORMNUD,AUTO_LINKLOCAL>
```

In this example, the following devices were displayed:

- `em0`: The Ethernet interface.
- `lo0`: The loop interface is a software loopback mechanism which may be used for performance analysis, software testing, and/or local communication. More information in lo(4).

The example shows that `em0` is up and running.

The key indicators are:

1. `UP` means that the interface is configured and ready.
2. The interface has an IPv4 Internet (`inet`) address, `192.168.1.19`.
3. The interface has an IPv6 Internet (`inet6`) address, `fe80::21f:16ff:fe0f:275a%em0`.
4. It has a valid subnet mask (`netmask`), where `0xffffff00` is the same as `255.255.255.0`.
5. It has a valid broadcast address, `192.168.1.255`.
6. The MAC address of the interface (`ether`) is `00:1f:16:0f:27:5a`.
7. The physical media selection is on autoselection mode (`media: Ethernet autoselect (1000baseT <full-duplex>)`).
8. The status of the link (`status`) is `active`, indicating that the carrier signal is detected. For `em0`, the `status: no carrier` status is normal when an Ethernet cable is not plugged into the interface.

If the ifconfig(8) output had shown something similar to the next output it would indicate the interface has not been configured:

```
em0: flags=8822<BROADCAST,SIMPLEX,MULTICAST> metric 0 mtu 1500

options=481249b<RXCSUM,TXCSUM,VLAN_MTU,VLAN_HWTAGGING,VLAN_HWCSUM,LRO,WOL_MAGIC,VLAN_H
WFILTER,NOMAP>
        ether 00:1f:16:0f:27:5a
        media: Ethernet autoselect
        status: no carrier
        nd6 options=29<PERFORMNUD,IFDISABLED,AUTO_LINKLOCAL>
```

7.3.1. Configuring Static IPv4 Address

This section provides a guide to configuring a static IPv4 address on a FreeBSD system.

The network interface card configuration can be performed from the command line with ifconfig(8) but will not persist after a reboot unless the configuration is also added to /etc/rc.conf.



If the network was configured during installation by bsdinstall(8), some entries for the network interface card (NICs) may be already present. Double check /etc/rc.conf before executing sysrc(8).

The IP address can be set executing the following command:

```
# ifconfig em0 inet 192.168.1.150/24
```

To make the change persist across reboots execute the following command:

```
# sysrc ifconfig_em0="inet 192.168.1.150 netmask 255.255.255.0"
```

Add the default router executing the following command:

```
# sysrc defaultrouter="192.168.1.1"
```

Add the DNS records to /etc/resolv.conf:

```
nameserver 8.8.8.8
nameserver 8.8.4.4
```

Then restart `netif` and `routing` executing the following command:

```
# service netif restart && service routing restart
```

The connection can be tested using ping(8):

```
% ping -c2 www.FreeBSD.org
```

The output should be similar to the following:

```
PING web.geo.FreeBSD.org (147.28.184.45): 56 data bytes
64 bytes from 147.28.184.45: icmp_seq=0 ttl=51 time=55.173 ms
64 bytes from 147.28.184.45: icmp_seq=1 ttl=51 time=53.093 ms

--- web.geo.FreeBSD.org ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 53.093/54.133/55.173/1.040 ms
```

7.3.2. Configuring Dynamic IPv4 Address

If the network has a DHCP server, it is very easy to configure the network interface to use DHCP. FreeBSD uses dhclient(8) as the DHCP client. dhclient(8) will automatically provide the IP, the netmask and the default router.

To make the interface work with DHCP execute the following command:

```
# sysrc ifconfig_em0="DHCP"
```

dhclient(8) can be used manually by running the following command:

```
# dhclient em0
```

The output should be similar to the following:

```
DHCPREQUEST on em0 to 255.255.255.255 port 67
DHCPACK from 192.168.1.1
unknown dhcp option value 0x7d
bound to 192.168.1.19 -- renewal in 43200 seconds.
```

In this way it can be verified that the address assignment using DHCP works correctly.



dhclient(8) client can be started in background. This can cause trouble with applications depending on a working network, but it will provide a faster startup in many cases.

To execute dhclient(8) in background execute the following command:

```
# sysrc background_dhclient="YES"
```

Then restart `netif` executing the following command:

```
# service netif restart
```

The connection can be tested using ping(8):

```
% ping -c2 www.FreeBSD.org
```

The output should be similar to the following:

```
PING web.geo.FreeBSD.org (147.28.184.45): 56 data bytes
64 bytes from 147.28.184.45: icmp_seq=0 ttl=51 time=55.173 ms
64 bytes from 147.28.184.45: icmp_seq=1 ttl=51 time=53.093 ms

--- web.geo.FreeBSD.org ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 53.093/54.133/55.173/1.040 ms
```

7.3.3. IPv6

IPv6 is the new version of the well-known IP protocol, also known as IPv4.

IPv6 provides several advantages over IPv4 as well as many new features:

- Its 128-bit address space allows for 340,282,366,920,938,463,463,374,607,431,768,211,456 addresses. This addresses the IPv4 address shortage and eventual IPv4 address exhaustion.
- Routers only store network aggregation addresses in their routing tables, thus reducing the average space of a routing table to 8192 entries. This addresses the scalability issues associated with IPv4, which required every allocated block of IPv4 addresses to be exchanged between Internet routers, causing their routing tables to become too large to allow efficient routing.
- Address autoconfiguration (RFC4862).
- Mandatory multicast addresses.
- Built-in IPsec (IP security).
- Simplified header structure.
- Support for mobile IP.
- IPv6-to-IPv4 transition mechanisms.

FreeBSD includes the KAME project IPv6 reference implementation and comes with everything needed to use IPv6.

This section focuses on getting IPv6 configured and running.

There are three different types of IPv6 addresses:

Unicast

A packet sent to a unicast address arrives at the interface belonging to the address.

Anycast

These addresses are syntactically indistinguishable from unicast addresses but they address a group of interfaces. The packet destined for an anycast address will arrive at the nearest interface.

Multicast

These addresses identify a group of interfaces. A packet destined for a multicast address will arrive at all interfaces belonging to the multicast group. The IPv4 broadcast address, usually `xxx.xxx.xxx.255`, is expressed by multicast addresses in IPv6.

When reading an IPv6 address, the canonical form is represented as `x:x:x:x:x:x:x:x`, where each `x` represents a 16 bit hex value. An example is `FEBC:A574:382B:23C1:AA49:4592:4EFE:9982`.

Often, an address will have long substrings of all zeros. A `::` (double colon) can be used to replace one substring per address. Also, up to three leading `0`s per hex value can be omitted. For example, `fe80::1` corresponds to the canonical form `fe80:0000:0000:0000:0000:0000:0000:0001`.

A third form is to write the last 32 bits using the well known IPv4 notation. For example, `2002::10.0.0.1` corresponds to the hexadecimal canonical representation `2002:0000:0000:0000:0000:0000:0a00:0001`, which in turn is equivalent to `2002::a00:1`.

To view a FreeBSD system's IPv6 address execute the following command:

```
# ifconfig
```

The output should be similar to the following:

```
em0: flags=8863<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500

options=481249b<RXCSUM,TXCSUM,VLAN_MTU,VLAN_HWTAGGING,VLAN_HWCSUM,LRO,WOL_MAGIC,VLAN_H
WFILTER,NOMAP>
        ether 00:1f:16:0f:27:5a
        inet 192.168.1.150 netmask 0xffffff00 broadcast 192.168.1.255
        inet6 fe80::21f:16ff:fe0f:275a%em0 prefixlen 64 scopeid 0x1
        media: Ethernet autoselect (1000baseT <full-duplex>)
        status: active
        nd6 options=23<PERFORMNUD,ACCEPT_RTADV,AUTO_LINKLOCAL>
```

In this example, the `em0` interface is using `fe80::21f:16ff:fe0f:275a%em0`, an auto-configured link-local address which was automatically generated from the MAC address.

Some IPv6 addresses are reserved. A list of reserved addresses can be checked in the following table:

Table 10. Example IPv6 Reserved Addresses

IPv6 address	Description	Notes
`::/128`	unspecified	Equivalent to `0.0.0.0` in IPv4.
`::1/128`	loopback address	Equivalent to `127.0.0.1` in IPv4.
`::ffff:0.0.0.0/96`	IPv4 mapped IPv6 address	The lower 32 bits are the IPv4 address for compatibility with IPv4 hosts and routers.
`fe80::/10`	link-local unicast	Equivalent to 169.254.0.0/16 in IPv4.
`fc00::/7`	unique-local	Unique local addresses are intended for local communication and are only routable within a set of cooperating sites.
`ff00::/8`	multicast	
`2000::/3`	global unicast	All global unicast addresses are assigned from this pool. The first 3 bits are `001`.
`2001:db8::/32, 3fff::/20`	documentation	IPv6 address prefix for use in documentation.

For further information on the structure of IPv6 addresses, refer to RFC4291.

7.3.4. Configuring Static IPv6 Address

To configure a FreeBSD system as an IPv6 client with a static IPv6 address it is necessary to set the IPv6 address.

Execute the following commands to meet the requirements:

```
# sysrc ifconfig_em0_ipv6="inet6 2001:db8:4672:6565:2026:5043:2d42:5344 prefixlen 64"
```

To assign a default router, specify its address executing the following command:

```
# sysrc ipv6_defaultrouter="2001:db8:4672:6565::1"
```

To configure an additional IPv6 anycast address, specify the anycast address as an `_aliasN`, as specified in rc.conf(5), followed by the `anycast` option:

```
# sysrc ifconfig_em0_alias0="inet6 2001:db8:4672:6565::a anycast"
```

Keep in mind that the applications can't bind to anycast addresses; in that case you need to use an alias address instead.

7.3.5. Configuring Dynamic IPv6 Address

To dynamically configure the IPv6 address of the interface using SLAAC, execute the following commands:

```
# sysrc ifconfig_em0_ipv6="inet6 accept_rtadv"
# sysrc rtsold_enable="YES"
```

Note that when IPv6 packet forwarding is enabled (i.e., `ipv6_gateway_enable=YES`), the system will not configure a SLAAC address unless the `net.inet6.ip6.rfc6204w3` sysctl(8) variable is set to 1.

7.3.6. Router Advertisement and Host Auto Configuration

This section demonstrates how to setup rtadvd(8) on an IPv6 router to advertise the IPv6 network prefix and default route.

To enable rtadvd(8), execute the following command:

```
# sysrc rtadvd_enable="YES"
```

It is important to specify the interface on which to do IPv6 router advertisement. For example, to tell rtadvd(8) to use `em0`:

```
# sysrc rtadvd_interfaces="em0"
```

Next, create the configuration file, /etc/rtadvd.conf as seen in this example:

```
em0:\
    :addrs#1:addr="2001:db8:1f11:246::":prefixlen#64:tc=ether:
```

Replace `em0` with the interface to be used and `2001:db8:1f11:246::` with the prefix of the allocation.

For a dedicated `/64` subnet, nothing else needs to be changed. Otherwise, change the `prefixlen#` to the correct value.

7.3.7. IPv6 and IPv4 Address mapping

When IPv6 is enabled on a server, there may be a need to enable IPv4 mapped IPv6 address communication. This compatibility option allows for IPv4 addresses to be represented as IPv6

addresses. Permitting IPv6 applications to communicate with IPv4 and vice versa may be a security issue.

This option may not be required in most cases and is available only for compatibility. This option will allow IPv6-only applications to work with IPv4 in a dual stack environment. This is most useful for third party applications which may not support an IPv6-only environment.

To enable this feature execute the following command:

```
# sysrc ipv6_ipv4mapping="YES"
```

7.4. Wireless Networks

Most wireless networks are based on the IEEE® 802.11 standards.

FreeBSD supports networks that operate using 802.11a, 802.11b, 802.11g and 802.11n.



802.11ac support on FreeBSD is currently under development.

A basic wireless network consists of multiple stations communicating with radios that broadcast in either the 2.4GHz or 5GHz band, though this varies according to the locale and is also changing to enable communication in the 2.3GHz and 4.9GHz ranges.

There are three basic steps to configure a wireless network:

1. Scan and select an access point
2. Authenticate the station
3. Configure an IP address or use DHCP.

The following sections discuss each step.

7.4.1. Quick Start to Connect to a Wireless Network

Connecting FreeBSD to an existing wireless network is a very common situation.

This Quick Start procedure shows the steps required to connect to a network using basic authentication. A more detailed procedure is given in the next section.

The first step is to obtain the Service Set Identifier (SSID) and Pre-Shared Key (PSK) for the wireless network from the network administrator.

The second step is to add an entry for this network to /etc/wpa_supplicant.conf.

If the file does not exist, create it. Consult wpa_supplicant.conf(5) for details on formatting entries in this file.

```
ctrl_interface=/var/run/wpa_supplicant
eapol_version=1
```

```
ap_scan=1
fast_reauth=1

network={
 ssid="myssid" ①
 psk="mypsk" ②
}
```

① Is the SSID of the wireless network. Replace it with the name of the wireless network.

② Is the PSK of the wireless network. Replace it with the password of the wireless network.

The third step is to add the network interface entry to configure the network on startup. Obtain the network device interface by using the sysctl `net.wlan.devices`. In the listing below, the output of this sysctl shows the network device interface is "iwm0".

```
# sysctl net.wlan.devices
```

The output should be similar to the following:

```
net.wlan.devices: iwm0
```

In the sysrc line that follows, replace "iwm0" with the output of the sysctl command if needed.

```
# sysrc wlans_iwm0="wlan0"
# sysrc ifconfig_wlan0="WPA DHCP"
```

- And the last step is to restart the `netif` service executing the following command:

```
# service netif restart
```

7.4.2. Basic Wireless Configuration

This section provides a more detailed example of configuring a wireless network card. To find out what wireless network cards are in the system check the section Identify Network Adapters.

```
# ifconfig wlan0 create wlandev iwm0
```

To make the change persist across reboots execute the following command:

```
# sysrc wlans_iwm0="wlan0"
```



Since the regulatory situation is different in various parts of the world, it is

necessary to correctly set the domains that apply to your location to have the correct information about what channels can be used.

The available region definitions can be found in /etc/regdomain.xml. To set the data at runtime, use `ifconfig`:

```
# ifconfig wlan0 regdomain etsi2 country AT
```

To persist the settings, add it to /etc/rc.conf:

```
# sysrc create_args_wlan0="country AT regdomain etsi2"
```

7.4.3. Scan Wireless Networks

Available wireless networks can be scanned using ifconfig(8).

To list the wireless networks execute the following command:

```
# ifconfig wlan0 up list scan
```

The output should be similar to the following:

```
SSID/MESH ID                    BSSID              CHAN RATE    S:N     INT CAPS
FreeBSD                         e8:d1:1b:1b:58:ae    1   54M  -47:-96   100 EP   RSN
BSSLOAD HTCAP WPS WME
NetBSD                          d4:b9:2f:35:fe:08    1   54M  -80:-96   100 EP   RSN
BSSLOAD HTCAP WPS WME
OpenBSD                         fc:40:09:c6:31:bd   36   54M  -94:-96   100 EPS
VHTPWRENV APCHANREP RSN WPS BSSLOAD HTCAP VHTCAP VHTOPMODE WME
GNU-Linux                       dc:f8:b9:a0:a8:e0   44   54M  -95:-96   100 EP   WPA
RSN WPS HTCAP VHTCAP VHTOPMODE WME VHTPWRENV
Windows                         44:48:b9:b3:c3:ff   44   54M  -84:-96   100 EP
BSSLOAD VHTPWRENV HTCAP WME RSN VHTCAP VHTOPMODE WPS
MacOS                           46:48:b9:b3:c3:ff   44   54M  -84:-96   100 EP
BSSLOAD VHTPWRENV HTCAP WME RSN VHTCAP VHTOPMODE WPS
```

1. SSID/MESH ID identifies the name of the network.
2. BSSID identifies the MAC address of the access point.
3. CAPS field identifies the type of each network and the capabilities of the stations operating there (see the definition of `list scan` in ifconfig(8) for more details).

7.4.4. Connection and Authentication to a Wireless Network

Once a wireless network has been selected from the list of scanned networks, it is necessary to

perform the connection and the authentication. In the vast majority of wireless networks, authentication is done with a password configured in the router. Other schemes require cryptographic handshakes to be completed before data traffic can flow, either using pre-shared keys or secrets, or more complex schemes that involve backend services such as RADIUS.

7.4.4.1. Authenticate with WPA2/WPA/Personal

The authentication process in a wireless network is managed by wpa_supplicant(8).

The wpa_supplicant(8) configuration will be made in the /etc/wpa_supplicant.conf file. For more information, see wpa_supplicant.conf(5).

Once the scanning of the wireless networks has been carried out, a network has been chosen and have the password (PSK), that information will be added to the file /etc/wpa_supplicant.conf as in the following example:

```
network={
        scan_ssid=1 ①
        ssid="FreeBSD" ②
        psk="12345678" ③
}
```

① SSID scan technique. Only need to use this option if the network is hidden.

② Network name.

③ Password of the wireless network.

The next step will be to configure the wireless connection in the file /etc/rc.conf.

To use a static address it will be necessary to execute the following command:

```
# sysrc ifconfig_wlan0="inet 192.168.1.20 netmask 255.255.255.0"
```

To use a dynamic address it will be necessary to execute the following command:

```
# sysrc ifconfig_wlan0="WPA DHCP"
```

Then restart the network executing the following command:

```
# service netif restart
```



More information on how to perform more advanced methods of authentication can be obtained at Wireless Advanced Authentication.

7.4.4.2. Authenticate with Open Networks



It is important that the user is **very** careful when connecting to open networks without any kind of authentication.

Once the wireless network scan is done and the SSID of the wireless network is selected, execute the following command:

```
# ifconfig wlan0 ssid SSID
```

And then execute dhclient(8) to get the address configured:

```
# dhclient wlan0
```

7.4.5. Using Both Wired and Wireless Connections

A wired connection provides better performance and reliability, while a wireless connection provides flexibility and mobility. Laptop users typically want to roam seamlessly between the two types of connections.

On FreeBSD, it is possible to combine two or even more network interfaces together in a "failover" fashion. This type of configuration uses the most preferred and available connection from a group of network interfaces, and the operating system switches automatically when the link state changes.

Link aggregation and failover is covered in Link Aggregation and Failover and an example for using both wired and wireless connections is provided at Failover Mode Between Ethernet and Wireless Interfaces.

7.5. Hostname

The hostname represents the fully qualified domain name (FQDN) of the host on the network.



If no hostname is set for the host, FreeBSD will call itself `Amnesiac`.

7.5.1. Check The Current Hostname

hostname(1) can be used to check the current hostname:

```
$ hostname
```

The output should be similar to the following:

```
freebsdhostname.example.com
```

7.5.2. Change Hostname

To change the hostname of the host and persist it across reboots execute the following command:

```
# sysrc hostname="freebsdhostname.example.com"
```

7.6. DNS

The DNS could be understood as a telephone directory in which an IP is identified to a hostname and vice versa.

There are three files that handle how a FreeBSD system interact with the DNS. These three files are hosts(5), resolv.conf(5) and nsswitch.conf(5)

Unless otherwise stated in the /etc/nsswitch.conf file, FreeBSD will look at the addresses in the /etc/hosts file and then the DNS information in the /etc/resolv.conf file.



The nsswitch.conf(5) file specifies how the nsdispatch (name-service switch dispatcher) should operate.

By default, the hosts section of the /etc/nsswitch.conf file will be as follows:

```
hosts: files dns
```

For example, in case of using the nscd(8) service. The order of preference could be changed by leaving the line as follows:

```
hosts: files cache dns
```

7.6.1. Local addresses

The /etc/hosts file is a simple text database who provide host name to IP address mappings. Entries for local computers connected via a LAN can be added to this file for simplistic naming purposes instead of setting up a DNS server. Additionally, /etc/hosts can be used to provide a local record of Internet names, reducing the need to query external DNS servers for commonly accessed names.

For example, in the case of having a local instance of www/gitlab-ce in a local environment, it could be added as follows to the file /etc/hosts:

```
192.168.1.150 git.example.com git
```

7.6.2. Configuring the Nameserver

How a FreeBSD system accesses the Internet Domain Name System (DNS) is controlled by

resolv.conf(5).

The most common entries to /etc/resolv.conf are:

`nameserver`	The IP address of a name server the resolver should query. The servers are queried in the order listed with a maximum of three.
`search`	Search list for hostname lookup. This is normally determined by the domain of the local hostname.
`domain`	The local domain name.

A typical /etc/resolv.conf looks like this:

```
search example.com
nameserver 147.11.1.11
nameserver 147.11.100.30
```



Only one of the `search` and `domain` options should be used.

When using DHCP, dhclient(8) usually rewrites /etc/resolv.conf with information received from the DHCP server.



If the machine in which the configuration is being made is **not** a DNS server, local-unbound(8) can be used to improve DNS lookup performance.

To enable it at boot time execute the following command:

```
# sysrc local_unbound_enable="YES"
```

To start the local-unbound(8) service execute the following command:

```
# service local_unbound start
```

7.7. Troubleshooting

When troubleshooting hardware and software configurations, check the simple things first.

- Is the network cable plugged in?
- Are the network services properly configured?
- Is the firewall configured correctly?
- Is the NIC supported by FreeBSD?
- Is the router working correctly?



Before sending a bug report, always check the Hardware Notes in the FreeBSD release page, update the version of FreeBSD to the latest STABLE version, check the mailing list archives, and search the Internet.

7.7.1. Troubleshooting in Wired Networks

If the card works, yet performance is poor, read through tuning(7). Also, check the network configuration as incorrect network settings can cause slow connections.

`No route to host` messages occur if the system is unable to route a packet to the destination host. This can happen if no default route is specified or if a cable is unplugged. Check the output of `netstat -rn` and make sure there is a valid route to the host. If there is not, read Gateways and Routes.

`ping: sendto: Permission denied` error messages are often caused by a misconfigured firewall. If a firewall is enabled on FreeBSD but no rules have been defined, the default policy is to deny all traffic, even ping(8). Refer to Firewalls for more information.

7.7.2. Troubleshooting in Wireless Networks

This section describes a number of steps to help troubleshoot common wireless networking problems.

- If the access point is not listed when scanning, check that the configuration has not limited the wireless device to a limited set of channels.
- If the device cannot associate with an access point, verify that the configuration matches the settings on the access point. This includes the authentication scheme and any security protocols. Simplify the configuration as much as possible. If using a security protocol such as WPA2 or WPA, configure the access point for open authentication and no security to see if traffic will pass.
- Once the system can associate with the access point, diagnose the network configuration using tools like ping(8).
- There are many lower-level debugging tools. Debugging messages can be enabled in the 802.11 protocol support layer using wlandebug(8).

Part II: Common Tasks

Now that the basics have been covered, this part of the book discusses some frequently used features of FreeBSD. These chapters:

- Introduce popular and useful desktop applications: browsers, productivity tools, document viewers, and more.
- Introduce a number of multimedia tools available for FreeBSD.
- Explain the process of building a customized FreeBSD kernel to enable extra functionality.
- Describe the print system in detail, both for desktop and network-connected printer setups.
- Show how to run Linux applications on the FreeBSD system.

Some of these chapters recommend prior reading, and this is noted in the synopsis at the beginning of each chapter.

Chapter 8. Desktop Environments

8.1. Synopsis

While FreeBSD is popular as a server for its performance and stability, it is also well suited for day-to-day use as a desktop. With over 36000 applications available in the FreeBSD ports tree, it is straightforward to build a customized desktop that can run a wide variety of desktop applications. This chapter demonstrates how to install popular desktop environments as well as desktop applications such as web browsers, productivity software, document viewers, and financial software.

Prerequisites:

- Readers of this chapter should already understand how to install either the X Window System or Wayland on FreeBSD.
- Readers are instructed throughout this chapter to install official packages. Refer to the section on using the ports collection to build customized packages from ports.

8.2. Desktop Environments

This section describes how to install and configure some popular desktop environments on a FreeBSD system. A desktop environment can range from a simple window manager to a complete suite of desktop applications.

Table 11. Supported desktop environments

Name	License	Package
Plasma	GPL 2.0 or later	x11/kde
GNOME	GPL 2.0 or later	x11/gnome
XFCE	GPL, LGPL, BSD	x11-wm/xfce4
MATE	GPL 2.0, LGPL 2.0	x11/mate
Cinnamon	GPL 2.0 or later	x11/cinnamon
LXQT	GPL, LGPL	x11-wm/lxqt

8.2.1. Plasma

Plasma is a easy-to-use desktop environment made by the KDE community. This desktop provides a suite of applications with a consistent look and feel, a standardized menu and toolbars, keybindings, color-schemes, internationalization, and a centralized, dialog-driven desktop configuration.

More information on Plasma can be found at the Plasma homepage. For FreeBSD-specific information, consult the FreeBSD homepage at KDE.

8.2.1.1. Install Plasma meta package

To install the Plasma desktop with a basic set of KDE Applications, execute:

```
# pkg install kde
```

Additional KDE Applications are tagged with the `kde-applications` category. Search for them by executing the following command:

```
% pkg search -S origin kde-applications
```

8.2.1.2. Minimal Plasma installation

To install a minimal Plasma desktop, execute:

```
# pkg install plasma6-plasma
```



This installation is **really** minimal. The Konsole terminal emulator must be installed separately executing:

```
# pkg install konsole
```

8.2.1.3. Configure Plasma

Plasma uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of Plasma.

Enable the D-Bus service in rc.conf(5) to start at system boot:

```
# sysrc dbus_enable="YES"
```

8.2.1.4. Start Plasma

The preferred way to start Plasma is automatically via the x11/sddm display manager. To install x11/sddm, execute:

```
# pkg install sddm
```

Enable the SDDM service in rc.conf(5) to start at system boot:

```
# sysrc sddm_enable="YES"
```

A keyboard language, in this case Spanish, can be set in SDDM with the following command:

```
# sysrc sddm_lang="es_ES"
```

Alternatively, start Plasma in an Xorg session is by manually invoking startx(1). For this to work, the following line is needed in ~/.xinitrc:

```
% echo "exec dbus-launch --exit-with-x11 ck-launch-session startplasma-x11" >
~/.xinitrc
```

Experimentally, start Plasma manually in a Wayland session by executing the following command:

```
% exec dbus-launch ck-launch-session startplasma-wayland
```

8.2.2. GNOME

GNOME is a user-friendly desktop environment. It includes a panel for starting applications and displaying status, a desktop, a set of tools and applications, and a set of conventions that make it easy for applications to cooperate and be consistent with each other.

8.2.2.1. Install GNOME meta package

To install the GNOME meta package with GNOME Desktop and Applications, execute:

```
# pkg install gnome
```

8.2.2.2. Minimal GNOME installation

To install the GNOME-lite meta package with a GNOME desktop slimmed down for only the basics, execute:

```
# pkg install gnome-lite
```

8.2.2.3. Configure GNOME

GNOME requires `/proc` to be mounted. Add this line to `/etc/fstab` to mount this file system automatically during system startup:

```
# Device                Mountpoint      FStype  Options         Dump    Pass#
proc                    /proc           procfs  rw              0       0
```

GNOME uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of GNOME.

Enable D-BUS service in /etc/rc.conf to start at system boot:

```
# sysrc dbus_enable="YES"
```

8.2.2.4. Start GNOME

GNOME Display Manager is the preferred display manager for GNOME. GDM is installed as part of the GNOME package.

Enable GDM in /etc/rc.conf to start at system boot:

```
# sysrc gdm_enable="YES"
```

A second method to start GNOME is by manually invoking startx(1). For this to work, the following line is needed in ~/.xinitrc:

```
% echo "exec gnome-session" > ~/.xinitrc
```

8.2.3. XFCE

XFCE is a desktop environment based on the GTK+, lightweight and provides a simple, efficient, easy-to-use desktop. It is fully configurable, has a main panel with menus, applets, and application launchers, provides a file manager and sound manager, and is themeable. Since it is fast, light, and efficient, it is ideal for older or slower machines with memory limitations.

8.2.3.1. Install XFCE

To install the XFCE meta package, execute:

```
# pkg install xfce
```

8.2.3.2. Configure XFCE

XFCE uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of XFCE.

Enable D-BUS in /etc/rc.conf to start at system boot:

```
# sysrc dbus_enable="YES"
```

8.2.3.3. Start XFCE

x11/lightdm is a display manager that supports different display technologies and is a good choice as it is very lightweight, requires little memory usage, and has fast performance.

To install it, execute:

```
# pkg install lightdm lightdm-gtk-greeter
```

Enable lightdm in `/etc/rc.conf` to start at system boot:

```
# sysrc lightdm_enable="YES"
```

A second method to start XFCE is by manually invoking startx(1). For this to work, the following line is needed in `~/.xinitrc`:

```
% echo '. /usr/local/etc/xdg/xfce4/xinitrc' > ~/.xinitrc
```

8.2.4. MATE

The MATE Desktop Environment is the continuation of GNOME 2. It provides an intuitive and attractive desktop environment using traditional metaphors.

8.2.4.1. Install MATE meta package

To install the MATE meta package that includes the MATE Desktop with some extra applications such as text editor, archiver manager, etc., execute:

```
# pkg install mate
```

8.2.4.2. Minimal MATE installation

To install the MATE lite meta package with MATE desktop slimmed down for only the basics, execute:

```
# pkg install mate-base
```

8.2.4.3. Configure MATE

MATE requires `/proc` to be mounted. Add this line to `/etc/fstab` to mount this file system automatically during system startup:

```
# Device                Mountpoint      FStype  Options         Dump    Pass#
proc                    /proc           procfs  rw              0       0
```

MATE uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of MATE. Enable D-BUS in `/etc/rc.conf` to start at system boot:

```
# sysrc dbus_enable="YES"
```

8.2.4.4. Start MATE

x11/lightdm is a display manager that supports different display technologies and is a good choice as it is very lightweight, requires little memory usage, and has fast performance.

To install it, execute:

```
# pkg install lightdm lightdm-gtk-greeter
```

Enable lightdm in `/etc/rc.conf` to start at system boot:

```
# sysrc lightdm_enable="YES"
```

A second method to start MATE is by manually invoking startx(1). For this to work, the following line is needed in `~/.xinitrc`:

```
% echo "exec dbus-launch --exit-with-x11 ck-launch-session mate-session" > ~/.xinitrc
```

8.2.5. Cinnamon

Cinnamon is a UNIX® desktop which provides advanced innovative features and a traditional user experience. The desktop layout is similar to Gnome 2. The underlying technology is forked from Gnome Shell. The emphasis is put on making users feel at home and providing them with an easy to use and comfortable desktop experience.

8.2.5.1. Install Cinnamon

To install the Cinnamon package, execute:

```
# pkg install cinnamon
```

8.2.5.2. Configure Cinnamon

Cinnamon requires `/proc` to be mounted. Add this line to `/etc/fstab` to mount this file system automatically during system startup:

```
# Device                Mountpoint      FStype  Options         Dump    Pass#
proc                    /proc           procfs  rw              0       0
```

Cinnamon uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of Cinnamon. Enable D-BUS in `/etc/rc.conf` to start at

system boot:

```
# sysrc dbus_enable="YES"
```

8.2.5.3. Start Cinnamon

x11/lightdm is a display manager that supports different display technologies and is a good choice as it is very lightweight, requires little memory usage, and has fast performance.

To install it execute:

```
# pkg install lightdm lightdm-gtk-greeter
```

Enable lightdm in `/etc/rc.conf` to start at system boot:

```
# sysrc lightdm_enable="YES"
```

A second method to start Cinnamon is by manually invoking startx(1). For this to work, the following line is needed in `~/.xinitrc`:

```
% echo "exec dbus-launch --exit-with-x11 ck-launch-session cinnamon-session" >
~/.xinitrc
```

8.2.6. LXQT

LXQt is an advanced, easy-to-use, and fast desktop environment based on Qt technologies. It has been tailored for users who value simplicity, speed, and an intuitive interface. Unlike most desktop environments, LXQt also works fine with less powerful machines.

8.2.6.1. Install LXQT

To install the LXQT meta package, execute:

```
# pkg install lxqt
```

8.2.6.2. Configure LXQT

LXQT requires `/proc` to be mounted. Add this line to `/etc/fstab` to mount this file system automatically during system startup:

```
# Device                Mountpoint      FStype  Options         Dump    Pass#
proc                    /proc           procfs  rw              0       0
```

LXQT uses dbus-daemon(1) for a message bus and hardware abstraction. This application is automatically installed as a dependency of LXQT.

Enable D-BUS in `/etc/rc.conf` to start at system boot:

```
# sysrc dbus_enable="YES"
```

8.2.6.3. Start LXQT

The preferred LXQT display manager is x11/sddm. To install x11/sddm, execute:

```
# pkg install sddm
```

Enable SDDM service in `/etc/rc.conf` to start at system boot:

```
# sysrc sddm_enable="YES"
```

The keyboard language can be set in SDDM by running the following command (for example, for Spanish):

```
# sysrc sddm_lang="es_ES"
```

A second method to start LXQT is by manually invoking startx(1). For this to work, the following line is needed in `~/.xinitrc`:

```
% echo "exec dbus-launch --exit-with-x11 ck-launch-session startlxqt" > ~/.xinitrc
```

8.3. Browsers

This section describes how to install and configure some popular web browsers on a FreeBSD system, from full web browsers with high resource consumption to command line web browsers with reduced resource usage.

Table 12. Supported browsers

Name	License	Package	Resources Needed
Firefox	MPL 2.0	www/firefox	Heavy
Chromium	BSD-3 and others	www/chromium	Heavy
Iridium browser	BSD-3 and others	www/iridium-browser	Heavy
Falkon	MPL 2.0	www/falkon-qtonly	Heavy
Konqueror	GPL 2.0 or later	x11-fm/konqueror	Medium

Name	License	Package	Resources Needed
Gnome Web (Epiphany)	GPL 3.0 or later	www/epiphany	Medium
qutebrowser	GPL 3.0 or later	www/qutebrowser	Medium
Dillo	GPL 3.0 or later	www/dillo2	Light
Links	GPL 2.0 or later	www/links	Light
w3m	MIT	www/w3m	Light

8.3.1. Firefox

Firefox is an open source browser that features a standards-compliant HTML display engine, tabbed browsing, popup blocking, extensions, improved security, and more. Firefox is based on the Mozilla codebase.

To install the package of the latest release version of Firefox, execute:

```
# pkg install firefox
```

To instead install Firefox Extended Support Release (ESR) version, execute:

```
# pkg install firefox-esr
```

8.3.2. Chromium

Chromium is an open source browser project that aims to build a safer, faster, and more stable web browsing experience. Chromium features tabbed browsing, popup blocking, extensions, and much more. Chromium is the open source project upon which the Google Chrome web browser is based.

To install Chromium, execute:

```
# pkg install chromium
```



The executable for Chromium is /usr/local/bin/chrome, not /usr/local/bin/chromium.

8.3.3. Iridium browser

Iridium is a free, open, and libre browser modification of the Chromium code base, with privacy being enhanced in several key areas. Automatic transmission of partial queries, keywords, metrics to central services is inhibited and only occurs with consent.

To install Iridium, execute:

```
# pkg install iridium-browser
```

8.3.4. Falkon

Falkon is a new-ish and very fast QtWebEngine browser. It aims to be a lightweight web browser available on all major platforms. Falkon has all standard functions someone can expect from a web browser. It includes bookmarks, history (both also in sidebar) and tabs. Beyond that, AdBlock plugin can block ads, Click2Flash can block Flash content and SSL Manager can edit the local CA Certificates database.

To install Falkon, execute:

```
# pkg install falkon
```

8.3.5. Konqueror

Konqueror is more than a web browser as it is also a file manager and a multimedia viewer. It supports WebKit, a rendering engine used by many modern browsers including Chromium, as well as its own KHTML engine.

To install Konqueror, execute:

```
# pkg install konqueror
```

8.3.6. Gnome Web (Epiphany)

Gnome Web (Epiphany) is a web browser designed to be as lightweight and fast as possible, at the expense of many of the features found in other browsers.

To install Gnome Web (Epiphany), execute:

```
# pkg install epiphany
```

8.3.7. qutebrowser

Qutebrowser is a keyboard-focused browser with a minimal GUI. It is based on Python and PyQt5 and free software, licensed under the GPL.

To install qutebrowser, execute:

```
# pkg install qutebrowser
```

8.3.8. Dillo

Dillo aims to be a multiplatform alternative browser that is small, stable, developer-friendly, usable, fast, and extensible. This new, experimental version of Dillo is based upon FLTK toolkit, rather than GTK1, and has been substantially rewritten.

To install Dillo, execute:

```
# pkg install dillo
```

8.3.9. Links

A lynx-like web browser with text and graphics modes with many features like displaying tables, menus, etc.

To install Links, execute:

```
# pkg install links
```

8.3.10. w3m

w3m is a pager/text-based web browser. It is a similar application to Lynx, but it has several features Lynx does not have like rendering tables and rendering frames.

To install w3m, execute:

```
# pkg install w3m
```

8.4. Development tools

This section describes how to install and configure some popular development tools on a FreeBSD system.

Table 13. Supported development tools

Name	License	Package	Resources Needed
Visual Studio Code	MIT	editors/vscode	Heavy
Qt Creator	QtGPL	devel/qtcreator	Heavy
Kdevelop	GPL 2.0 or later and LGPL 2.0 or later	devel/kdevelop	Heavy
Eclipse IDE	EPL	java/eclipse	Heavy
Vim	VIM	editors/vim	Light
Neovim	Apache 2.0	editors/neovim	Light

Name	License	Package	Resources Needed
GNU Emacs	GPL 3.0 or later	editors/emacs	Light

8.4.1. Visual Studio Code

Visual Studio Code is a type of tool that combines the simplicity of a code editor with what developers need for their core edit-build-debug cycle. It provides comprehensive editing and debugging support, an extensibility model, and lightweight integration with existing tools.

To install Visual Studio Code, execute:

```
# pkg install vscode
```

8.4.2. Qt Creator

Qt Creator is a cross-platform IDE (integrated development environment) tailored to the needs of Qt developers. Functionalities included with Qt Creator are:

- code editor with C++, QML and ECMAscript support;
- rapid code navigation tools;
- static code checking and style hints when typing;
- context sensitive help;
- visual debugger;
- integrated GUI layout and forms designer.

To install Qt Creator, execute:

```
# pkg install qtcreator
```

8.4.3. kdevelop

Open source, feature-full, plugin extensible IDE for C/C++ and other programming languages. It is based on KDevPlatform and the KDE and Qt libraries, and it has been under development since 1998.

To install kdevelop, execute:

```
# pkg install kdevelop
```

8.4.4. Eclipse IDE

The Eclipse Platform is an open extensible IDE for anything and yet nothing in particular. The Eclipse Platform provides building blocks and a foundation for constructing and running integrated

software-development tools. The Eclipse Platform allows tool builders to independently develop tools that integrate with other people's tools.

To install Eclipse IDE, execute:

```
# pkg install eclipse
```

8.4.5. Vim

Vim is a highly configurable text editor built to enable efficient text editing. It is an improved version of the vi editor distributed with most UNIX systems.

Vim is often called a "programmer's editor," and so useful for programming that many consider it an entire IDE. It is not just for programmers, though. Vim is perfect for all kinds of text editing, from composing email to editing configuration files.

To install Vim, execute:

```
# pkg install vim
```

8.4.6. Neovim

Neovim is an aggressive refactor of editors/vim. It represents a complete overhaul of the codebase with many sanity improvements, including sensible defaults, a built-in terminal emulator, asynchronous plugin architecture, and powerful APIs designed for speed and extensibility. It retains full compatibility with almost all Vim plugins and scripts.

To install Neovim, execute:

```
# pkg install neovim
```

8.4.7. GNU Emacs

GNU Emacs is an extensible, customizable, free/libre text editor. At its core is an interpreter for Emacs Lisp, a dialect of the Lisp programming language with extensions to support text editing.

To install GNU Emacs, execute:

```
# pkg install emacs
```

8.5. Desktop office productivity

When it comes to productivity, users often look for an office suite or an easy-to-use word processor. While some desktop environments like KDE Plasma provide an office suite, there is no default

productivity package. Several office suites and graphical word processors are available for FreeBSD, regardless of the installed desktop environments.

This section demonstrates how to install the following popular productivity software and indicates if the application is resource-heavy, takes time to compile from ports, or has any major dependencies.

Table 14. Supported Desktop office productivity suites

Name	License	Package	Resources Needed
LibreOffice	MPL 2.0	editors/libreoffice	Heavy
Calligra Suite	LGPL and GPL	editors/calligra	Medium
AbiWord	GPL 2.0 or later	editors/abiword	Medium

8.5.1. LibreOffice

LibreOffice is a free software office suite developed by The Document Foundation. It is compatible with other major office suites and available on a variety of platforms. It is a rebranded fork of Apache OpenOffice and includes applications found in a complete office productivity suite: a word processor, spreadsheet, presentation manager, drawing program, database management program, and a tool for creating and editing mathematical formulæ. It is available in a number of different languages and internationalization has been extended to interfaces, spell checkers, and dictionaries. More information about LibreOffice can be found at libreoffice.org.

To install LibreOffice, execute:

```
# pkg install libreoffice
```

The LibreOffice package comes by default only in English. To have a localized version of LibreOffice it is necessary to install a language pack. For example, for the version localized in Spanish, it is necessary to install the package editors/libreoffice-es with the command:

```
# pkg install es-libreoffice
```

8.5.2. Calligra

The KDE Plasma desktop environment includes an office suite which can be installed separately from KDE Plasma. Calligra includes standard components that can be found in other office suites. Words is the word processor, Sheets is the spreadsheet program, Stage manages slide presentations, and Karbon is used to draw graphical documents.

To install Calligra, execute:

```
# pkg install calligra
```

8.5.3. AbiWord

AbiWord is a free word processing program similar in look and feel to Microsoft® Word. It is fast, contains many features, and is user-friendly.

AbiWord can import or export many file formats, including some proprietary ones like Microsoft® .rtf.

To install AbiWord, execute:

```
# pkg install abiword
```

8.6. Document Viewers

Some new document formats have gained popularity since the advent of UNIX® and the viewers they require may not be available in the base system. This section demonstrates how to install the following document viewers:

Table 15. Supported Document Viewers

Name	License	Package	Resources Needed
Okular	GPL 2.0	graphics/okular	Heavy
Evince	GPL 2.0	graphics/evince	Medium
ePDFView	GPL 2.0	graphics/epdfview	Medium
Xpdf	GPL 2.0	graphics/xpdf	light
Zathura	Zlib	graphics/zathura	light

8.6.1. Okular

Okular is a universal document viewer, part of the KDE Plasma project.

Okular combines excellent functionality with the versatility of supporting different kind of documents, like PDF, Postscript, DjVu, CHM, XPS, ePub and others.

To install Okular, execute:

```
# pkg install okular
```

8.6.2. Evince

Evince is a document viewer for multiple document formats including PDF and Postscript. Part of the GNOME project. The goal of evince is to replace document viewers such as ggv and gpdf with a single, simple application.

To install Evince, execute:

```
# pkg install evince
```

8.6.3. ePDFView

ePDFView is a lightweight PDF document viewer that only uses the Gtk+ and Poppler libraries. The aim of ePDFView is to make a simple PDF document viewer, similar to Evince but without using the GNOME libraries.

To install ePDFView, execute:

```
# pkg install epdfview
```

8.6.4. Xpdf

For users that prefer a small FreeBSD PDF viewer, Xpdf provides a light-weight and efficient viewer which requires few resources. It uses the standard X fonts and does not require any additional toolkit.

To install Xpdf, execute:

```
# pkg install xpdf
```

8.6.5. Zathura

Zathura is a highly customizable and functional document viewer. It provides a minimalistic and space saving interface as well as an easy usage that mainly focuses on keyboard interaction.

To install zathura, with PDF support, execute:

```
# pkg install zathura zathura-pdf-mupdf
```

Additionally, one can install graphics/zathura-pdf-poppler for alternative PDF support, graphics/zathura-ps for PostScript support, graphics/zathura-djvu for DjVu support, and graphics/zathura-cb for comic book support.

8.7. Finance

For managing personal finances on a FreeBSD desktop, some powerful and easy-to-use applications can be installed. Some are compatible with widespread file formats, such as the formats used by Quicken and Excel.

This section covers these programs:

Table 16. Supported Finance programs

Name	License	Package	Resources Needed
KMyMoney	GPL 2.0	finance/kmymoney	Heavy
GnuCash	GPL 2.0 and GPL 3.0	finance/gnucash	Heavy

8.7.1. KMyMoney

KMyMoney is a personal finance application created by the KDE community. KMyMoney aims to provide the important features found in commercial personal finance manager applications. It also highlights ease-of-use and proper double-entry accounting among its features. KMyMoney imports from standard Quicken QIF files, tracks investments, handles multiple currencies, and provides a wealth of reports.

To install KMyMoney, execute:

```
# pkg install kmymoney
```

8.7.2. GnuCash

GnuCash is part of the GNOME effort to provide user-friendly, yet powerful, applications to end-users. GnuCash can be used to keep track of income and expenses, bank accounts, and stocks. It features an intuitive interface while remaining professional.

GnuCash provides a smart register, a hierarchical system of accounts, and many keyboard accelerators and auto-completion methods. It can split a single transaction into several more detailed pieces. GnuCash can import and merge Quicken QIF files. It also handles most international date and currency formats.

To install GnuCash, execute:

```
# pkg install gnucash
```

Chapter 9. Multimedia

9.1. Synopsis

The multimedia chapter provides an overview of multimedia support on FreeBSD. Multimedia applications and technologies have become an integral part of modern computing, and FreeBSD provides robust and reliable support for a wide range of multimedia hardware and software. This chapter covers various multimedia components such as audio, video, and image processing. It also discusses various media formats and codecs, as well as tools and applications for multimedia creation and playback. Additionally, the chapter covers multimedia system configuration, troubleshooting, and optimization. FreeBSD provides a robust platform for multimedia work, suitable for both multimedia enthusiasts and professional content creators. This chapter aims to help get the most out of FreeBSD's multimedia capabilities, providing useful information and practical examples to help get started.

9.2. Setting Up the Sound Card

By default, FreeBSD will automatically detect the sound card used by the system. FreeBSD supports a wide variety of sound cards. The list of supported sound cards can be consulted in sound(4).



It is only necessary to load the sound card module if FreeBSD has not detected it correctly.

Where it is not known which sound card the system has, or which module to use, the `snd_driver` metadriver can be loaded by executing the following command:

```
# kldload snd_driver
```

Alternatively, to load the driver as a module at boot time, place the following line in /boot/loader.conf:

```
snd_driver_load="YES"
```

9.2.1. Testing Sound

To confirm the sound card is detected the following command can be executed:

```
% dmesg | grep pcm
```

The output should be similar to the following:

```
pcm0: <Conexant CX20561 (Hermosa) (Analog 2.0+HP/2.0)> at nid 26,22 and 24 on hdaa0
```